AEREALITH AI

ARCHITECTURE BIBLE

Your Digital Life, Intelligently Connected

Cloudflare-first reference architecture / v0.6 / 12 August 2026

A companion architecture for people, organizations, knowledge, integrations, automation, and AI.

Table of Contents

Nine architecture parts, seventy-plus implementation domains, and the diagrams that connect them.

Table of Contents - Continued

Document Control

DOCUMENT STRUCTURE

What this Bible is - and what it is not.

This document is the architectural north star for Aerealith AI. It records the product intent, vocabulary, system boundaries, data ownership rules, security posture, infrastructure selections, self-hosted escape hatches, and the operating principles that should remain stable even while implementations change.

Architecture diagrams: System Context; Cloud & Edge Platform; Data Ownership; Canonical Domain Model; Knowledge Garden; AI Control Loop; Extensions & Automation; API & Security; Operations & Delivery; Governance Loop.

STATUS	**Working architecture. Treat decisions marked REQUIRED as implementation constraints; treat RECOMMENDED items as defaults that may change through measured production evidence.**

- **Primary audience:** Aerealith maintainers, contributors, security reviewers, platform engineers, product designers, and future integration authors.
- **Scope:** web application, companion runtime, Knowledge Garden, connector/integration ecosystem, automation engine, admin/ops plane, storage, observability, security, cost controls, and portability.
- **Out of scope:** pixel-level frontend specifications, individual vendor contract terms, model-specific prompt text, and feature-by-feature user documentation.
- **Update rule:** architecture decisions change through an ADR. Do not silently replace a foundational decision in code.

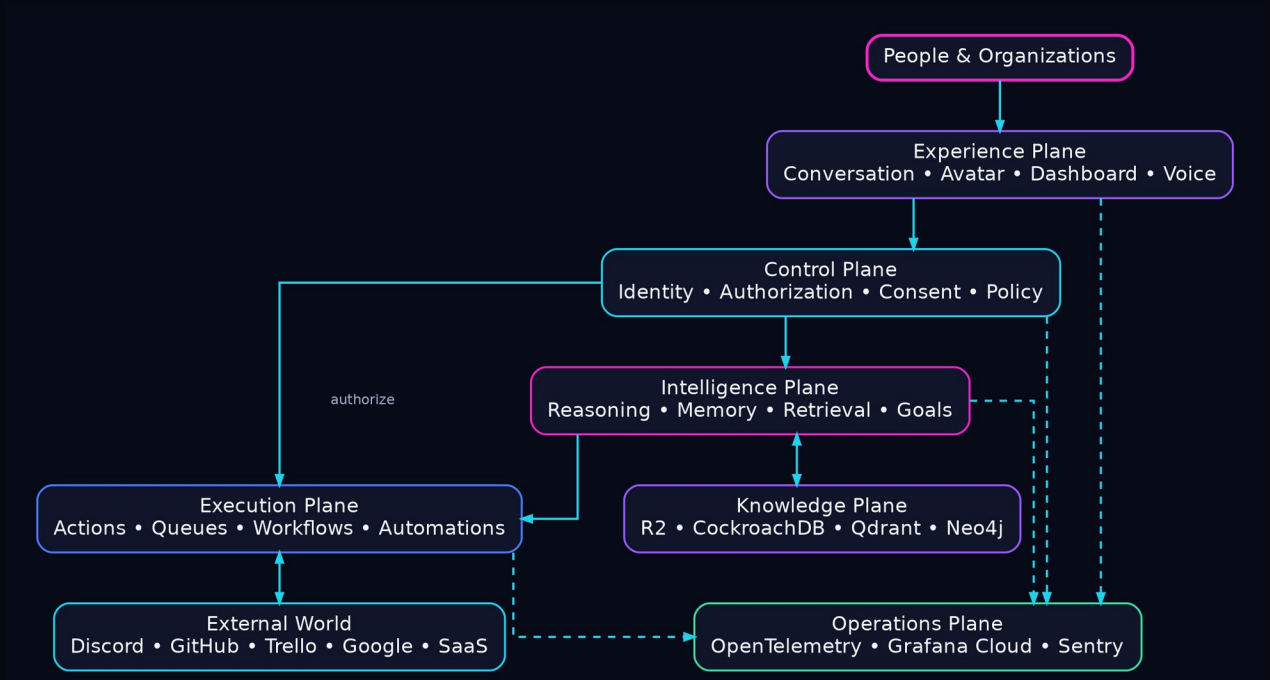
This edition is Cloudflare-first for compute, edge delivery and orchestration: Cloudflare Workers, R2, KV, Queues, Workflows, Durable Objects, Flagship, AI Gateway and Workers AI form the managed edge/platform default. PostgreSQL-compatible relational state is accessed through Drizzle; DEC-004 defines PostgreSQL as the default baseline and CockroachDB as an explicitly tested distributed-SQL target. Qdrant is the semantic/vector retrieval store and Neo4j is the relationship/knowledge-graph store. Resend handles transactional email. OpenTelemetry provides vendor-neutral instrumentation, Grafana Cloud is the primary operations/telemetry plane, and Sentry is the application error/debugging plane. Every external dependency remains behind a provider-neutral boundary with a documented self-hosted or portable path.

PART 01

Vision & Foundations

ARCHITECTURE VIEW

Why Aerealith exists, the vocabulary it uses, the architectural rules that constrain it, and the six-plane system model.



LEGEND Pink = human/companion-facing intelligence; cyan = control/trust and external boundaries; blue = execution; purple = knowledge; green = operations. Solid arrows are primary flows; dashed arrows are telemetry/observation; two-headed arrows are bidirectional exchange.

Product North Star

Augment human life without adding another layer of complexity.

Aerealith is not meant to be another dashboard users must learn. It is a conversational companion that becomes the interface to a person's digital world. A user should be able to sign up, meet Aerealith, speak naturally, connect services as the conversation demands, and gradually grant the level of autonomy they are comfortable with.

- **Conversation first:** the avatar and voice/text conversation are the primary interface; forms and dashboards are supporting surfaces.
- **No setup wall:** initial configuration is guided conversationally. The system should explain what it needs, why it needs it, and what permission is being granted.
- **Progressive autonomy:** Aerealith starts by observing and suggesting. Repeated approved behaviors become candidates for automations. The user explicitly chooses when Aerealith may act without asking each time.
- **Interruptible intelligence:** long analyses are presented in bite-size conversational chunks. Users can interrupt, drill down, skip, summarize, or change direction.
- **Companion continuity:** memory, goals, relationships, preferences, and knowledge are preserved with provenance and controls, so Aerealith becomes more useful over time rather than forcing users to re-explain themselves.

NORTH STAR	**After an interaction, the user should understand more, have less friction, or have less work left to do.**

Architecture Principles

Rules that should survive framework and vendor changes.

1. Cloudflare first, not Cloudflare trapped. Prefer native Cloudflare primitives when they fit, but wrap external boundaries so the platform can be moved or self-hosted.
2. One system of record per kind of truth. PostgreSQL/CockroachDB via Drizzle owns authoritative transactional entities and authorization state; R2 owns source blobs and immutable artifacts; Qdrant owns the rebuildable semantic/vector index; Neo4j owns the rebuildable relationship graph. Grafana Cloud/Sentry own observability projections, never business authority. Specialized stores are derived from authoritative records and provenance.
3. Everything important has a stable ID, owner, tenant, provenance, visibility, lifecycle state, and audit trail.
4. Read-only access is safer than write access. Connectors and integrations are separate trust levels.
5. Automations are explicit capabilities, not accidental side effects. Idempotency and undo/compensation are designed before execution.
6. AI is a planner and interpreter, not the authority for permission checks, money movement, deletion, or identity.
7. Every expensive computation should be cacheable, reusable, or justified by freshness requirements.
8. Private data does not become public knowledge by implication. Public contribution is explicit, reversible where possible, and policy-controlled.
9. Observability is part of the product. Every user-visible action should be traceable across model calls, queues, workflows, integrations, and data writes.
10. Prefer standards: OpenAPI, OAuth 2.1/OIDC, OpenTelemetry, OpenFeature, MCP where appropriate, webhooks, JSON Schema, and portable storage formats.

Canonical Vocabulary

The words Aerealith uses internally and externally.

Connector: Read-only capability that imports or queries information from an external service.

Integration: Read/write capability that can take actions in an external service.

Plugin: Installable extension package that can contribute connectors, integrations, tools, schemas, UI surfaces, events, or skills.

Skill: Higher-level capability that combines reasoning with one or more platform tools. Example: Project Manager uses Trello + GitHub + calendar + knowledge.

Automation: Persistent, permissioned rule that triggers actions from schedules, events, conditions, or learned patterns.

Knowledge Garden: The living, provenance-aware corpus Aerealith can search and relate. It contains user-private, organization-private, and explicitly public knowledge.

Entity: Stable transactional object: user, organization, connection, document, goal, task, permission, automation, conversation, credential reference, etc.

Artifact: File or generated binary object stored in R2.

Memory: User- or organization-scoped retained context that influences future assistance.

Goal: A durable desired outcome that Aerealith can connect to tasks, knowledge, conversations, and recommendations.

Action: A single permission-checked write operation.

Run: One execution instance of a skill, workflow, automation, ingestion job, or action sequence.

System Map

The platform as six cooperating planes.

Experience	**Web, mobile, voice, avatar, notifications**	**Conversation, onboarding, admin, marketplace**
Control	Identity, permissions, policies, feature flags	CockroachDB, Flagship, Zero Trust
Intelligence	AI router, skills, memory, retrieval, goals	Workers, AI Gateway, Qdrant, Neo4j
Execution	Actions, queues, workflows, automations	Workers, Queues, Workflows, Durable Objects
Knowledge	Ingestion, provenance, files, graph, vectors	R2, CockroachDB, Qdrant, Neo4j
Operations	Telemetry, errors, audits, cost, billing	OpenTelemetry, Grafana, Sentry, admin cockpit

The key boundary is between intelligence and authority. The intelligence plane can decide what it wants to do; the control plane decides whether that action is legal and allowed; the execution plane performs it; the operations plane records what happened.

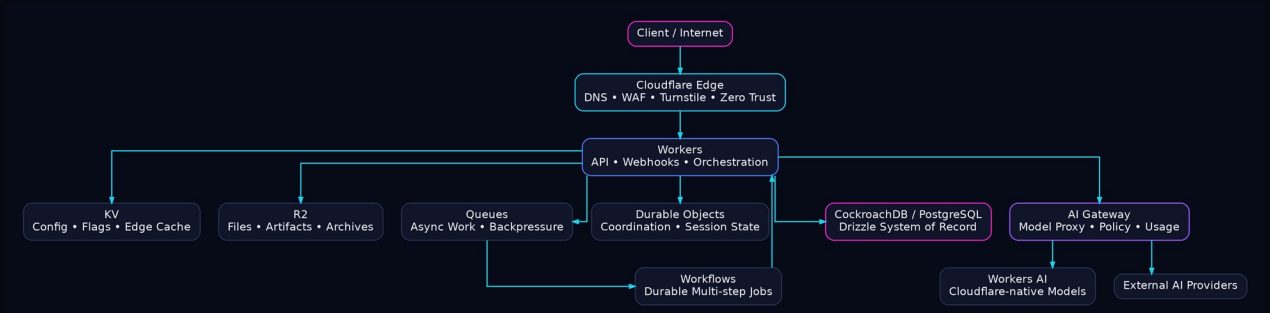
RULE	**No model response bypasses the policy/action gateway. Every external write is a typed action with explicit scopes.**

PART 02

Cloud & Edge Platform

ARCHITECTURE VIEW

Cloudflare-first runtime, storage, async execution, coordination, feature delivery, AI gatewaying, and edge security.



LEGEND Cyan = edge/security and coordination; blue = runtime/compute; purple = AI routing; pink = durable platform truth/user entry. Arrows show request, queue, workflow, storage, and model-call direction.

Cloudflare Workers

Primary edge compute and API runtime.

Use Workers for public APIs, AI orchestration, webhook receivers, lightweight transformation, permission checks, and service-to-service RPC. Keep handlers thin: validate, authorize, enqueue or invoke a durable primitive, and return.

CLOUD	**Cloudflare Workers - Primary edge compute and API runtime.** official
SELF-HOST	Self-hosted TypeScript/Node services on Kubernetes using Hono or Fastify behind Envoy/NGINX. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Cloudflare R2

Canonical blob/object store.

Store uploads, generated artifacts, normalized source snapshots, exports, backups, and optionally cold knowledge source material. Do not store large blobs in CockroachDB. Store the R2 object key, checksum, content type, size, encryption metadata, retention class, and provenance in CockroachDB.

CLOUD	**Cloudflare R2 - Canonical blob/object store.** official
SELF-HOST	MinIO for S3-compatible self-hosted object storage; Ceph RGW for very large infrastructure. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Cloudflare KV

Globally replicated read-heavy configuration and cache.

Use KV for non-authoritative configuration, public metadata, feature-adjacent cache entries, provider capabilities, low-risk lookup tables, and cache pointers. Do not use it for permission decisions that require immediate consistency or as the master copy of user entities.

CLOUD	**Cloudflare KV - Globally replicated read-heavy configuration and cache.** official
SELF-HOST	Valkey for high-speed key/value cache; Redis is also viable where licensing/hosting fits. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Cloudflare Queues

Asynchronous work and back-pressure.

Use Queues between request handling and expensive/slow work: ingestion, embedding, graph projection, crawler tasks, webhook fan-out, analytics events, outbound notifications, bulk sync, and retryable integration actions. Messages must be idempotent and contain stable run IDs.

CLOUD	**Cloudflare Queues - Asynchronous work and back-pressure.** official
SELF-HOST	NATS JetStream for lightweight eventing or RabbitMQ for explicit work queues and routing. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Cloudflare Workflows

Durable multi-step orchestration.

Use Workflows for stateful sequences that must survive retries and waits: OAuth connection setup, ingestion pipelines, long-running syncs, human approval, batch processing, connector backfills, export/delete jobs, and multi-step automations. Keep step payloads small; place artifacts in R2 and reference them by ID.

CLOUD	**Cloudflare Workflows - Durable multi-step orchestration.** official
SELF-HOST	Temporal for self-hosted durable execution and human-in-the-loop orchestration. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

2026 COST NOTE	**Workflows now has explicit billing dimensions for requests, CPU, state storage, and steps. Build workflows around meaningful durable boundaries instead of turning every function call into a workflow step.**

Durable Objects

Single-writer coordination and live session state.

Use Durable Objects selectively for conversational rooms, presence, live collaboration, rate/cohort coordination, WebSocket fan-out, per-user session state, and places where single-writer serialization simplifies correctness. Do not duplicate durable business truth that already belongs in CockroachDB.

CLOUD	**Durable Objects - Single-writer coordination and live session state.** official
SELF-HOST	Actor/stateful services on Kubernetes using Orleans/Akka, or NATS + a transactional datastore depending on workload. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Cloudflare Flagship

Feature flags and progressive rollout.

Flagship should gate beta functionality, provider rollouts, experiments, migration stages, and risky integrations. Use a naming convention such as aerealith.<domain>.<feature>. Never use a feature flag as an authorization check.

CLOUD	**Cloudflare Flagship - Feature flags and progressive rollout.** official
SELF-HOST	Unleash is the preferred self-hosted feature flag alternative and supports standardized rollout patterns. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Qdrant

Primary semantic retrieval and vector database for the Knowledge Garden.

Use Qdrant for embeddings, semantic search, recommendations, similarity retrieval, memory recall candidates and RAG retrieval. Prefer Qdrant Cloud while Aerealith is managed/cloud-first; retain the same API contract so Qdrant can later be self-hosted with Docker or Kubernetes. Store only derived vectors and compact retrieval metadata. Original source artifacts remain in R2, while authoritative ownership, tenancy, permissions, lifecycle and deletion state remain in the relational database. Qdrant metadata filters narrow the candidate set, but every protected source is re-authorized before content reaches the model or user.

CLOUD	**Qdrant Cloud - managed vector database for semantic search, RAG, recommendations and memory retrieval. https://qdrant.tech/cloud/**
SELF-HOST	Qdrant self-hosted - same engine deployed with Docker or Kubernetes. https://qdrant.tech/documentation/guides/installation/
WHY	Use the same logical adapter and collection/payload contract in cloud and self-hosted modes. Qdrant is derived retrieval state, not the source of truth or permission authority.

BOUNDARY	**Qdrant contains derived semantic vectors and retrieval metadata only. R2 preserves source artifacts; PostgreSQL/CockroachDB owns entities and authorization; Neo4j owns relationship traversal. Every protected retrieval is re-authorized against authoritative scope.**

Cloudflare AI Gateway

Unified model control and observability.

Route supported external model calls through AI Gateway for analytics, caching, rate limits, retries, fallback, and provider abstraction. Treat it as the outer AI traffic control plane, while Aerealith retains its own model policy and cost router.

CLOUD	**Cloudflare AI Gateway - Unified model control and observability.** official
SELF-HOST	LiteLLM Proxy provides an OpenAI-compatible multi-provider gateway and can be self-hosted. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Workers AI

Low-friction edge inference.

Use Workers AI for low-cost classification, embeddings or models that meet quality requirements. Do not force all AI workloads onto one provider. The model router selects the cheapest acceptable model based on capability, privacy, latency, and quality.

CLOUD	**Workers AI - Low-friction edge inference.** official
SELF-HOST	vLLM for production GPU inference; Ollama remains excellent for developer/local operation. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

Turnstile + Zero Trust

Abuse resistance and internal administration.

Turnstile protects signup, login recovery, public forms, and other abuse-prone endpoints. Zero Trust/Access protects internal administrative surfaces and engineering tools. Keep end-user application authorization in Aerealith itself.

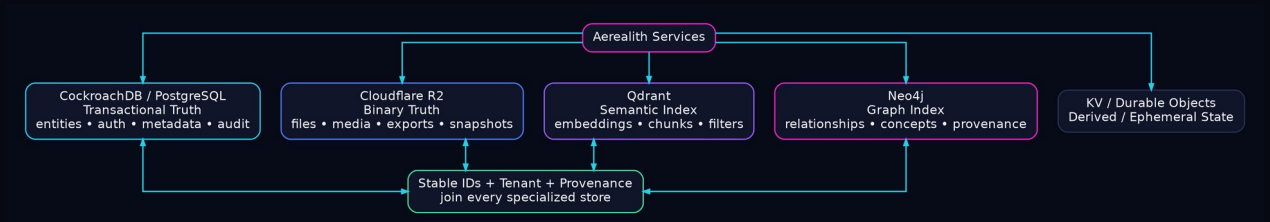
CLOUD	**Turnstile + Zero Trust - Abuse resistance and internal administration.** official
SELF-HOST	mCaptcha/ALTCHA for bot challenges; Authentik or Keycloak for self-hosted identity-aware access. project
WHY	Cloudflare-first default; maintain an abstraction boundary so the role can move without rewriting domain logic.

PART 03

Data & Storage Architecture

ARCHITECTURE VIEW

Authoritative transactional state, globally stable entities, consent-aware metadata, object storage, deduplication, and specialized data stores.



LEGEND Pink/cyan = authoritative application/transactional truth; blue = binary object truth; purple = rebuildable semantic/graph indexes; green = stable cross-store identity. Two-headed links mean references resolve both ways through stable IDs.

PostgreSQL / CockroachDB + Drizzle

Transactional source of truth with explicit CockroachDB compatibility.

Per DEC-004, PostgreSQL is the default relational baseline and Drizzle owns typed schemas, migrations and data access. CockroachDB is an accepted distributed-SQL deployment target only where compatibility validation passes. The database stores users, organizations, memberships, normalized roles and permissions, principal authorization versions, connection metadata, documents, goals, automations, audits, billing state and durable references to R2/Qdrant/Neo4j projections. Domain code never imports Drizzle or raw persistence records.

- Do store: stable IDs, ownership, tenant, normalized authorization state, lifecycle, timestamps, hashes, normalized metadata, durable references, policy decisions, audit facts and transactionally coupled state.
- Do not store: large file bodies, raw telemetry streams, embeddings, high-volume graph edges, provider secrets in plaintext, or temporary conversational token buffers.
- Consistency rule: authorization and billing read an authoritative strongly consistent path. Normalized role assignments are authoritative; the deprecated users.role projection must never be used for access decisions (DEC-016).
- **Multi-region rule:** start with the simplest topology that meets latency/compliance. Do not prematurely make every table global. Measure access patterns first.
- **Migration rule:** all schema changes are reviewed, forward-compatible where practical, and include rollback/repair guidance.

CLOUD	**CockroachDB Cloud - distributed SQL / PostgreSQL-compatible application database** official
SELF-HOST	Self-host CockroachDB where licensing/operations fit; PostgreSQL is the portability baseline for domain schemas. project
WHY	CockroachDB speaks the PostgreSQL wire protocol but is not identical to PostgreSQL. Keep compatibility differences covered by integration tests.

Canonical Domain Model

Bounded contexts, aggregate roots, and ownership rules for the platform.

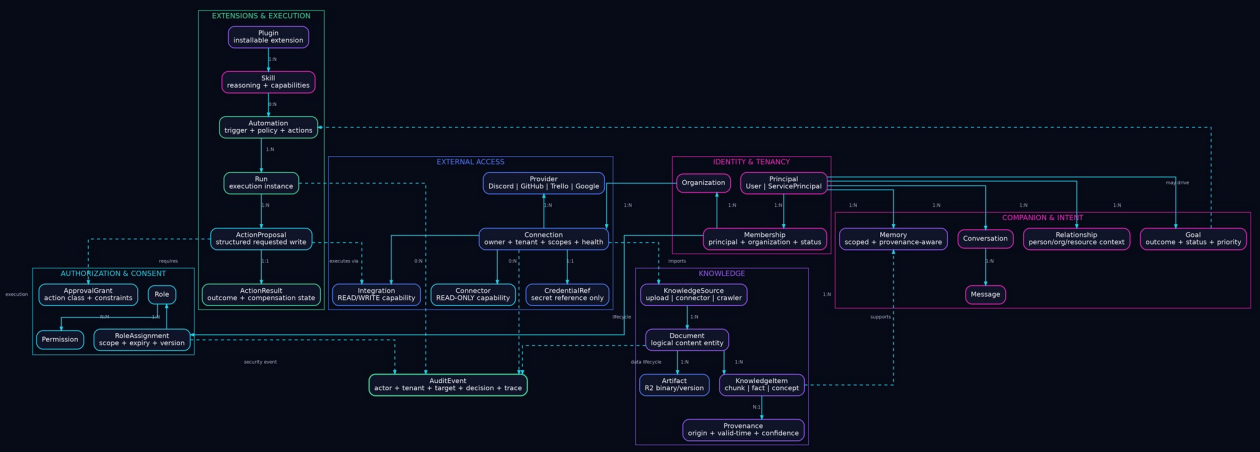
The domain model is provider-neutral and persistence-neutral. Domain entities express business meaning; Drizzle rows, Qdrant points, Neo4j nodes, and R2 objects are storage projections. The contexts below define where invariants live and which aggregate roots own changes.

Bounded context	Aggregate roots and responsibility
Identity & Tenancy	Principal (User or ServicePrincipal), Organization, Membership. Owns identity, tenant boundaries, membership lifecycle, and principal status.
Authorization & Consent	Role, Permission, RoleAssignment, ApprovalGrant. Owns exact permission+scope decisions, hierarchy, conflicts, expiry, revocation, and approved autonomy.
External Access	Provider, Connection, Connector, Integration, CredentialRef. Owns external-account lifecycle, scopes, health, read-only vs read/write capabilities, and secret references.
Knowledge	KnowledgeSource, Document, Artifact, KnowledgeItem, Provenance. Owns source identity, versions, visibility, classification, origin, valid-time, and rebuildable search/graph projections.
Companion & Intent	Conversation, Message, Memory, Goal, Relationship. Owns conversational continuity, retained context, user intent, goals, and relationship-aware assistance.
Extensions & Execution	Plugin, Skill, Automation, Run, ActionProposal, ActionResult. Owns installable capabilities, durable triggers, idempotent execution, retries, and compensation state.
Audit & Operations	AuditEvent, Notification, Incident, UsageRecord. Owns immutable security/action evidence, delivery state, operational incidents, usage and cost attribution.
Commerce & Entitlements	Plan, Subscription, Entitlement, UsageBudget. Owns commercial limits and feature eligibility; never substitutes for authorization.
UNIVERSAL ENTITY ENVELOPE	Every durable aggregate carries a stable id, tenant/owner where applicable, lifecycle status, visibility/classification, created/updated timestamps, version, provenance/retention references, and soft-delete marker. Vendor IDs are attributes - never Aerealith primary keys.

PART 03 / DATA & STORAGE

Domain Model Relationship Diagram

The canonical business model - separate from database tables and vendor-specific storage projections.



LEGEND **Pink:** identity, tenancy, companion intent | **Cyan:** authorization, consent, read-only trust boundaries | **Blue:** providers, connections, artifacts, write execution | **Purple:** knowledge, extensions, intelligence | **Green:** runs, outcomes, audit/operations

Solid arrows show aggregate relationships/ownership; labels show cardinality. Dashed arrows show policy, provenance, audit, or execution dependencies that do not transfer ownership.

DOMAIN RULE Domain objects never import Drizzle, provider SDKs, Qdrant, Neo4j, or R2 clients. Repositories/adapters map storage and provider representations into these domain types (DEC-006 / DEC-009).

Permissions & Consent

The trust architecture is a product feature.

Aerealith needs permission semantics that are understandable to non-technical users and enforceable by code. User-facing language can be simple while backend capabilities remain granular.

- **Observe:** read-only connector scopes. Aerealith can inspect and analyze but cannot modify the external service.
- **Ask every time:** write capability exists, but each sensitive action needs explicit approval.
- **Ask for a class of actions:** user approves a bounded automation or integration capability, such as moving Trello cards within one board.
- **Autonomous within policy:** Aerealith may execute approved actions while honoring limits, budgets, hours, target resources, and audit requirements.
- **Always blocked:** actions disallowed by organization policy, legal/compliance constraints, or security classification.

Permissions should compile into capability tokens or policy checks that name actor, tenant, integration, action, resource scope, constraints, expiry, and approval source. AI-generated text never substitutes for these checks.

UX RULE	**Every permission screen must answer: what can Aerealith see, what can it change, how long does permission last, and how can I turn it off?**

PART 03 / DATA & STORAGE

Storage Strategy

Active, knowledge, warm, archive, and recovery layers.

Layer	Primary store	Content
1. Active	R2 Standard + CockroachDB + live indexes	Current uploads, active sources, frequently used generated assets, transactional entities.
2. Knowledge	Qdrant + Neo4j + Cockroach metadata	Embeddings, semantic chunks, graph relations, provenance, freshness and confidence.
3. Warm	R2 with lifecycle rules / compressed snapshots	Older project artifacts, exports, inactive source snapshots that remain user-accessible.
4. Archive	R2 Infrequent Access where retrieval profile fits	Long-lived artifacts rarely read. Evaluate minimum storage duration/retrieval charges before moving.
5. Recovery	Independent backups / exports	Database backups, graph/vector rebuild manifests, infrastructure config, keys recovery procedures.

R2 is the default object store because Aerealith will repeatedly read user content during ingestion and retrieval. Cloudflare currently prices R2 without internet egress fees, which makes it attractive for an AI-heavy system that may process the same objects multiple times. Still, request costs and infrequent-access retrieval economics must be measured rather than ignored.

Blob Metadata in CockroachDB

R2 stores bytes; CockroachDB stores meaning and control.

The artifact table is the bridge between binary storage and every higher-level Aerealith entity. A document entity can reference one or more artifact versions; an R2 object should never need to carry permission logic in its key name.

- `artifact_id`
- `tenant_id`
- `owner_id`
- `r2_bucket`
- `r2_key`
- `sha256`
- `size_bytes`
- `mime_type`
- `filename`
- `classification`
- `visibility`
- `encryption_key_ref`
- `source_provider`
- `source_object_id`
- `source_etag`
- `source_modified_at`
- `ingest_status`
- `extraction_version`
- `retention_policy_id`
- `legal_hold`
- `created_at`
- `last_accessed_at`
- `deleted_at`

KEY RULE	**R2 keys should be opaque and stable, for example tenant/<tenant-id>/artifact/<artifact-id>/<version>. Never put email addresses, document titles, or secrets in keys.**

Access is brokered through authenticated Workers. Prefer short-lived signed access URLs only when direct upload/download is necessary. Every download decision should evaluate tenant membership, artifact visibility, retention/legal status, and any organization data policy.

Deduplication & Compression

Storage growth is controlled before archive tiers are needed.

Aerealith will ingest the same content through uploads, Drive, email attachments, knowledge imports, and shared organization sources. Deduplication is therefore a first-class storage feature.

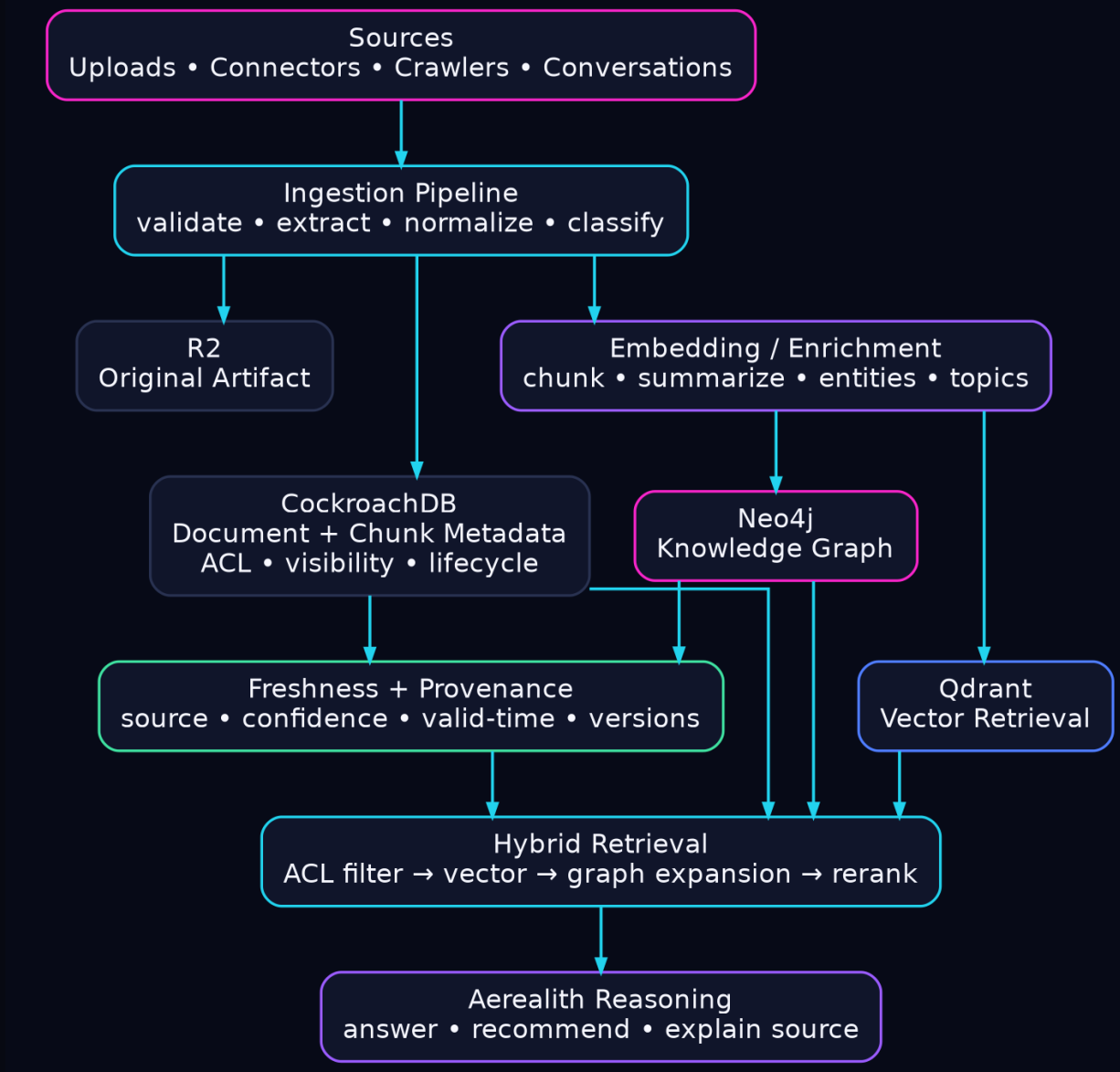
11. Compute a cryptographic content hash during upload/ingestion. Use a streaming hash so large files do not need to be buffered in memory.
12. Keep a tenant-scoped dedupe index by default. Cross-tenant physical dedupe may leak existence information and complicate deletion guarantees; only consider it after a dedicated privacy review.
13. Store normalized extracted text separately from original binary when doing so saves repeated extraction. Compress text and JSON with a deterministic versioned codec.
14. For revised documents, store a new artifact version and optionally delta-compress large machine-generated text snapshots. Never sacrifice the ability to reconstruct the user-visible source.
15. Deduplicate embeddings by normalized chunk hash + embedding model/version. A model change creates a new embedding generation, not an in-place overwrite.
16. Garbage collect unreachable derivative artifacts only after reference counting, retention, audit, and legal-hold checks.

PART 04

Knowledge Garden & Retrieval

ARCHITECTURE VIEW

How Aerealith ingests, versions, verifies, relates, retrieves, and governs private, organizational, and public knowledge.



LEGEND Pink = sources/reasoning endpoints; cyan = ingestion/retrieval control; purple = enrichment/graph knowledge; blue = vector retrieval; green = freshness/provenance verification. Arrows show ingestion then retrieval flow.

Knowledge Garden

A living corpus, not a folder of embeddings.

The Knowledge Garden is the durable intelligence substrate of Aerealith. It joins source material, extracted facts, semantic chunks, entities, relationships, freshness, confidence, and provenance. It is not the model weights and it is not a single database.

- **Private garden:** knowledge visible only to one user or personal tenant.
- **Organization garden:** knowledge shared according to organization roles and source permissions.
- **Shared/public contribution:** content a user explicitly marks public and is legally permitted to contribute.
- **Web-derived knowledge:** crawler-derived material with source URL, capture time, license/usage policy, and freshness schedule.
- **System knowledge:** Aerealith product docs, schemas, policies, tool descriptions, integration contracts, and approved operational runbooks.

CORE IDEA	**The graph tells Aerealith how things relate. The vector index tells it what is semantically similar. CockroachDB tells it who owns it and whether it may be used. R2 preserves the source.**

Ingestion Pipeline

Source -> artifact -> extraction -> chunks -> embeddings -> graph -> verification.

17. Receive source event or upload and create an ingestion run.
18. Resolve tenant, actor, visibility, consent, retention, and source permissions before parsing.
19. Persist or reference the original source in R2 and record immutable source metadata.
20. Extract text/structure with deterministic parsers; use OCR only for image/scanned material.
21. Normalize into a canonical document model and produce stable chunk IDs.
22. Classify content, detect entities, and generate embeddings through the model router.
23. Upsert vector points to Qdrant with tenant/visibility metadata, stable source and chunk IDs, embedding model/version, data classification, timestamps and provenance references.
24. Project entities/relationships into Neo4j using idempotent graph mutations.
25. Write indexing status and derived metadata into CockroachDB.
26. Run quality checks: extraction completeness, source coverage, permission inheritance, and retrievability.

Each step is independently retryable. Large intermediate output lives in R2. Workflow state contains IDs and compact status, not megabytes of document text. Queue messages are safe to replay.

Provenance, Freshness & Versioning

Never silently replace knowledge.

When something changes on the web or in a connected source, Aerealith should not erase history and pretend the new state was always true. It should version facts and source snapshots while exposing the current valid view to normal retrieval.

- **Provenance record:** source type, source locator, source object/version, captured_at, extracted_by, extraction version, model version, confidence, tenant/visibility, and evidence span.
- **Freshness policy:** static, event-driven, hourly, daily, weekly, monthly, manual, or source-specific TTL.
- **Supersession:** new evidence can supersede a fact while preserving previous fact state and validity interval.
- **Conflict:** multiple current sources can disagree. Store the conflict; do not average it into fake certainty.
- **Deletion:** if a private source is deleted and retention allows deletion, derivatives must be discoverable through provenance and removed/rebuilt.
- **Rebuildability:** vector and graph stores are derived indexes. A manifest must make them reconstructable from authorized sources and Cockroach metadata.

Qdrant

Managed Qdrant Cloud by default; self-host the same engine on Docker/Kubernetes when operational or sovereignty requirements justify it.

Qdrant is the semantic retrieval engine. Store embeddings and retrieval metadata, not authority. Each point carries tenant, visibility, source ID, document ID, chunk ID, classification, language, timestamps, embedding generation and provenance IDs. Collections are organized by embedding model/use case rather than one collection per user. Application-level authorization remains mandatory after vector retrieval.

CLOUD	**Qdrant Cloud - managed semantic/vector database. https://qdrant.tech/cloud/**
SELF-HOST	Qdrant self-hosted - Docker/Kubernetes deployment using the same engine and API. https://qdrant.tech/documentation/guides/installation/
WHY	This is not a dual-vendor abstraction: Qdrant is the vector technology. The portability decision is managed Qdrant Cloud versus self-hosted Qdrant, preserving identical domain semantics.

- Collection strategy: use a small number of collections partitioned by embedding model/use case, then payload filters for tenant, visibility, lifecycle and classification. Avoid one collection per user unless scale measurements prove it necessary.
- Embedding migration: record embedding model and generation. Re-embedding runs write to a new generation/collection or compatible namespace, validate recall/quality, then cut over with rollback available.
- Security: never rely on the vector database alone for authorization. Filter by tenant and allowed visibility, then re-check source authorization through the centralized authorization service before returning sensitive content.
- Cost controls: cache embeddings, hash chunks to avoid duplicate embedding, batch writes, tune HNSW/search parameters, cap candidate counts and archive/delete vectors when source retention expires.

Neo4j

Navigable relationship model and Knowledge Garden graph.

Neo4j stores the relationship projection: people, organizations, projects, goals, concepts, documents, events, tasks, services, integrations, and their typed connections. CockroachDB remains authoritative for identities, permissions, lifecycle, and transactional state; Neo4j is optimized for traversing relationships.

CLOUD	**Neo4j AuraDB - managed property graph database** official
SELF-HOST	Neo4j Community/self-managed where features/licensing meet requirements; Memgraph is an alternative property-graph engine. project
WHY	Use graph queries when traversal depth or relationship semantics would be awkward in relational joins. Do not copy every relational row into the graph.

- **Node identity:** Aerealith stable ID as a property; vendor-specific graph IDs are never external references.
- **Edge semantics:** typed, directional, time-aware relationships with provenance where inferred from source material.
- **Inference:** mark model-inferred edges separately from source-explicit edges and attach confidence/evidence.
- **Projection:** graph mutations are replayable from domain events / ingestion manifests so corruption does not destroy source truth.

Graph Ontology

Start small, typed, and explainable.

Do not attempt to model the universe on day one. The ontology should grow from retrieval and product needs. Every edge type needs a plain-language meaning, direction, permitted node types, provenance behavior, and deletion behavior.

NODES	Person, Organization, Project, Goal, Task, Conversation, Document, Concept, Event, Location, Service, Account, Connector, Integration, Automation, Skill, Plugin, Artifact, Source, Decision

EDGES	MEMBER_OF, OWNS, WORKS_ON, SUPPORTS_GOAL, DEPENDS_ON, MENTIONS, DERIVED_FROM, ABOUT, RELATED_TO, CREATED_BY, ASSIGNED_TO, CONNECTED_TO, USES, TRIGGERED_BY, SUPERSEDES, CONTRADICTS, EVIDENCE_FOR, SHARED_WITH

For user experience, relationships can be surfaced conversationally: “This email appears related to your Aerealith launch goal and the Trello epic you created last week. Should I link them?” A confirmed link becomes explicit user-curated knowledge rather than an opaque model guess.

Retrieval Architecture

Hybrid search before generation.

Aerealith retrieval combines deterministic filters, lexical search, Qdrant semantic vectors, Neo4j graph context, recency, source authority, provenance and user intent. “RAG” is a retrieval policy, not a raw vector dump into a prompt.

27. Resolve tenant, actor, conversation, goal and current task context.
28. Apply authorization and data classification filters before retrieval.
29. Generate query variants only when the request benefits from expansion.
30. Search semantic candidates in Qdrant and structured metadata/lexical indexes as appropriate. Qdrant payload filters are a first pass; source-level authorization is re-checked through @aerealith-ai/authorization before protected content is returned.
31. Traverse Neo4j for related people, projects, goals, decisions and source dependencies.
32. Rank candidates using relevance, freshness, authority, explicit user preference, and diversity of sources.
33. Fetch source excerpts from the canonical extracted artifact only for selected candidates.
34. Assemble a bounded context pack with provenance IDs and token budget.
35. Generate answer/action plan.
36. Attach source references internally so Aerealith can explain where a claim came from and later revalidate it.

Public vs Private Corpus

Contribution is explicit, provenance-aware, and revocable by policy.

Users may upload a document and choose private, organization-shared, or public contribution. Public contribution means Aerealith may use the content in the shared knowledge corpus according to terms and policy; it should not automatically mean the content is used to train foundation model weights.

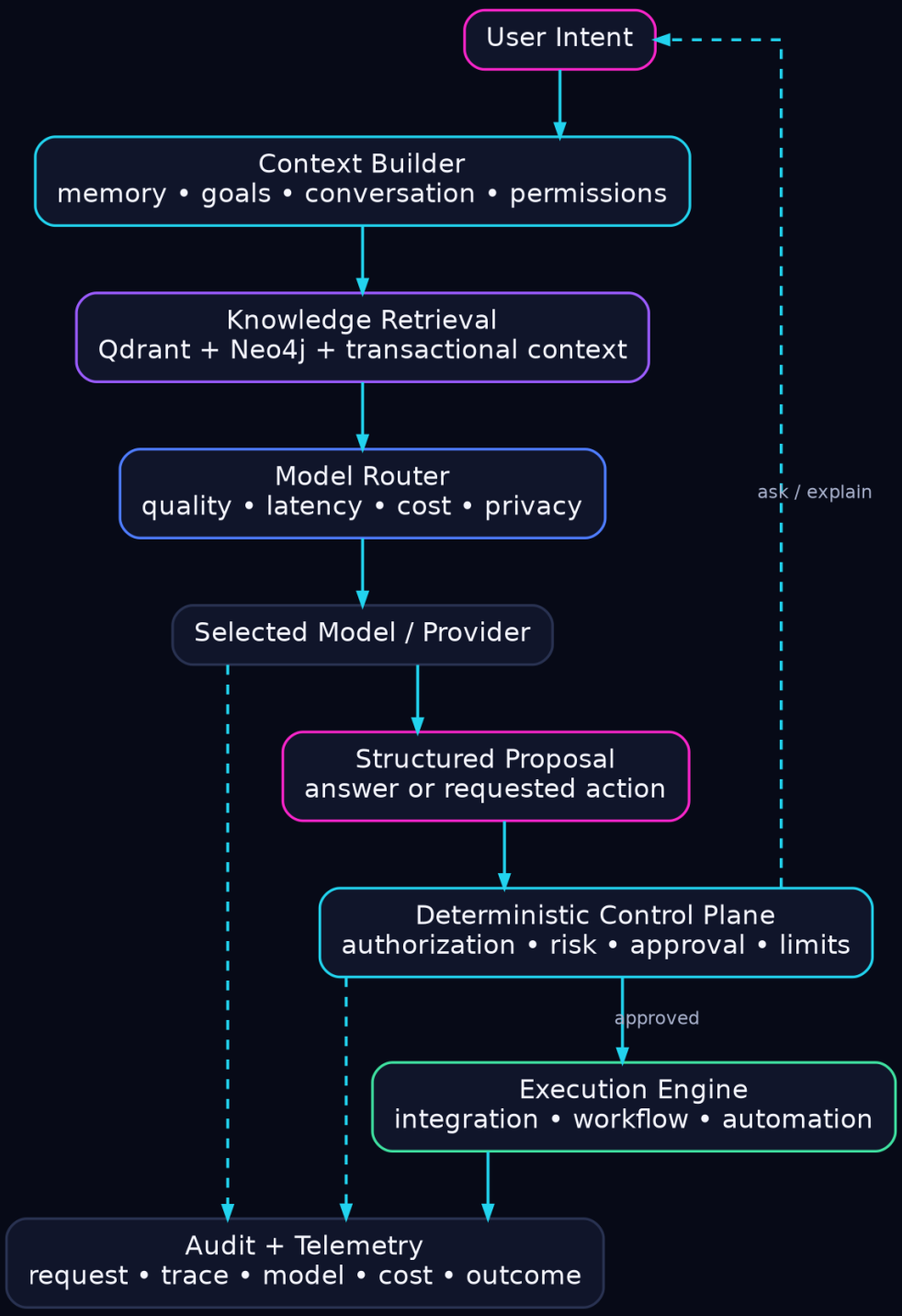
- **Private by default:** new uploads and connector-derived data remain private unless the user intentionally changes visibility.
- **Rights check:** public contribution UI requires the user to attest they have the right to contribute the content.
- **PII/sensitive review:** automated scanning can warn about likely sensitive information before public contribution.
- **Corpus separation:** keep public and private retrieval indexes logically separated even if the infrastructure is shared.
- **Training separation:** if Aerealith later performs model fine-tuning/training from donated content, that is a separate consent and dataset-governance program.
- **Withdrawal:** track dataset membership/provenance so future corpus builds can exclude withdrawn content; be transparent about limitations once content has been distributed or used in irreversible training.

PART 05

AI & Companion Intelligence

ARCHITECTURE VIEW

The reasoning runtime: model routing, cost controls, conversation, memory, goal awareness, and learned automation suggestions.



LEGEND Pink = user intent/proposal; cyan = context and deterministic policy; purple = retrieval; blue = model routing; green = approved execution. Dashed paths mean ask/explain or telemetry rather than direct authority.

AI Orchestration

One companion, many specialized capabilities.

Aerealith presents one coherent companion while internally routing work among models, deterministic code, retrieval systems, tools, and specialist agents. The user should not have to care which model solved a problem unless they ask or a policy requires disclosure.

- **Planner:** turns intent into an explicit plan or direct answer depending on complexity.
- **Retriever:** builds knowledge context with authorization and provenance.
- **Tool selector:** chooses typed skills/actions rather than free-form API calls.
- **Policy gate:** evaluates permission, risk, budget, and organization policy before action.
- **Executor:** runs tools through Workers/Queues/Workflows with idempotency.
- **Verifier:** checks result, side effects, citations/provenance, and whether compensation is needed.
- **Narrator:** turns state into concise, interruptible conversation.

RULE	**Use deterministic code for what can be deterministic. Do not pay an LLM to parse a UUID, enforce a permission, increment a quota, or decide whether a database transaction committed.**

Model Router

Optimize for acceptable quality, not maximum model size.

Aerealith should maintain a provider-neutral model registry. Every model is described by capability, context window, structured output support, tool use, latency class, privacy/data policy, geographic availability, reliability, and estimated cost.

- Tier 0: deterministic / cached answer - no model call.
- Tier 1: small/cheap model for classification, extraction, routing, short rewrite, lightweight summarization.
- Tier 2: balanced general model for most assistant conversation and moderate reasoning.
- Tier 3: premium reasoning/coding model for high-value complex work.
- Tier 4: specialist model for vision, audio, OCR-adjacent, embeddings, reranking, or domain-specific tasks.

AI Gateway handles traffic-level controls such as analytics, caching, fallback and rate limiting. Aerealith's router still owns business-level routing: user plan, monthly AI budget, task importance, privacy requirements, required modality, model health, and quality thresholds.

AI Cost Controls

Hundreds of calls per user are a design bug unless they buy real value.

- **Budget envelope:** track cost per user, tenant, conversation, skill, automation, provider, and model.
- **Semantic cache:** cache stable answers/results by normalized input + authorized knowledge version, not just raw prompt text.
- **Derivative cache:** store OCR, extraction, chunking, summaries, entity extraction, embeddings, and reranking results using versioned keys.
- **Batching:** batch embeddings and bulk metadata operations.
- **Prompt discipline:** retrieve only needed context; do not resend entire conversations/documents.
- **Escalation:** cheap model attempts first only when escalation can be verified; do not create latency by blindly cascading every request.
- **BYOK / enterprise routing:** support organization-owned provider keys later without letting secrets reach client code.
- **Quotas:** soft warning before hard limits; critical user actions should degrade gracefully rather than mysteriously fail.

METRIC	Track cost per successful user outcome, not only cost per token. A cheap model that causes retries can be more expensive.

Conversation Runtime

The primary interface is a state machine disguised as a natural conversation.

Conversation turns need durable metadata: participant(s), tenant, active goal, current topic, connected sources, pending approvals, active run IDs, interruptions, user pacing preferences, and tool results. The raw transcript can be stored separately from compact conversational state.

- **Interruptibility:** stream generation and maintain cancellation tokens. Tool execution that cannot be canceled must become a run whose completion can be reported later.
- **Silence awareness:** voice UX may detect a long pause and ask whether to continue. This behavior is a user preference and accessibility feature, not a fixed timer everywhere.
- **Bite-sized delivery:** large reports are converted into a sequence of conversational cards/segments with drill-down anchors.
- **Resume:** a conversation can continue after reconnect by loading active run state, recent compact summary, pending approvals, and relevant memory.
- **Multimodal:** voice, text, files, images, and structured UI all map into the same conversation event model.

Memory Profiles

Ephemeral, session, durable, and protected memory.

Not every observation deserves lifelong retention. Aerealith should classify memory by purpose and sensitivity.

Class	Behavior
Ephemeral	Used during one request; discarded after completion unless required for audit.
Session	Retained for the active conversation or short continuity window.
Durable preference	Stable user choices that improve future assistance, with explicit edit/delete controls.
Goal/project memory	Facts and decisions attached to an ongoing goal or project.
Relationship memory	User-curated context about people/organizations; higher privacy sensitivity.
Protected memory	Sensitive categories requiring additional policy, encryption, retention, and possibly no model-provider transmission.

Users need a “What Aerealith remembers about me” surface with search, provenance, correction, deletion, and memory-off controls.

Goal Awareness

Aerealith connects daily activity to durable outcomes.

Goals should be first-class entities rather than chat notes. A goal can have owner, description, target date, status, success criteria, importance, related projects, linked services, tasks, blockers, and evidence of progress.

- **Goal detection:** Aerealith may suggest creating a goal when repeated intent is clear, but does not silently create durable goals.
- **Linking:** incoming emails, Trello cards, GitHub issues, calendar events, documents, and conversations can be suggested as related.
- **Progress:** derive progress from evidence, not vibes. A board state or completed issue can be evidence; a model summary is interpretation.
- **Recommendations:** prioritize suggestions by user goals, urgency, commitments, and known constraints.
- **Drift:** if activity repeatedly conflicts with a stated goal, Aerealith can gently surface the mismatch without moralizing.
- **Trello:** Trello can be one execution surface for goals/epics, not the canonical definition of the goal itself.

Pattern Learning & Suggested Automations

Autonomy is earned through repeated consent.

Aerealith should detect recurring sequences such as “every morning summarize unread project mail,” “after a GitHub release update Trello and Discord,” or “I always archive this newsletter.” Pattern detection produces a suggestion, not an unannounced automation.

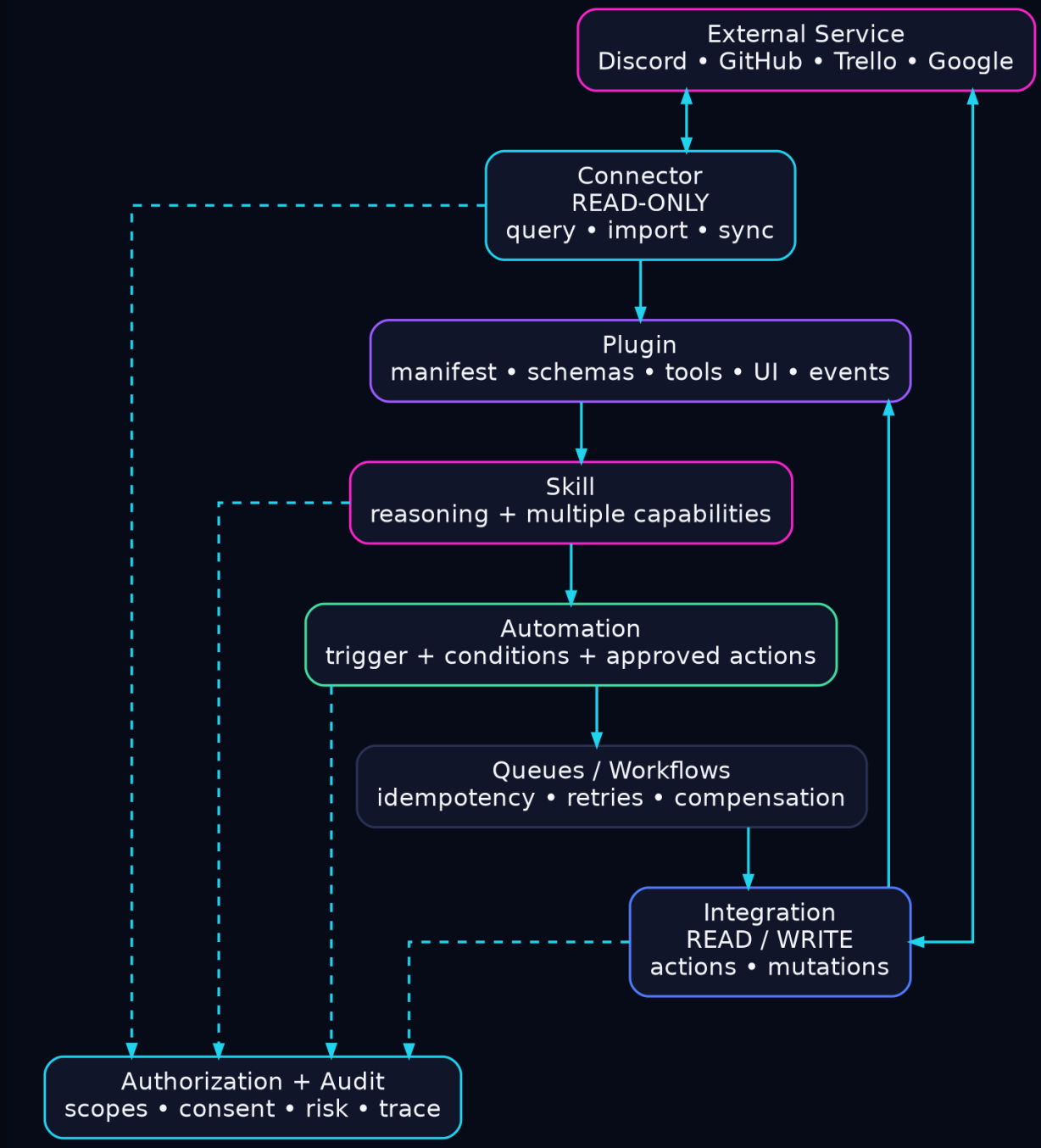
37. Observe repeated user-approved actions and their context.
38. Cluster patterns by trigger, conditions, actions, targets, timing, and exceptions.
39. Estimate confidence and potential benefit.
40. Present a plain-language proposed automation with examples of what it will and will not do.
41. Ask for explicit scope, limits, schedule, notification mode, and undo behavior.
42. Create a versioned automation definition and capability grant.
43. Run initially in preview/dry-run mode for higher-risk classes.
44. Track outcomes, exceptions, manual overrides, and user satisfaction.
45. Suggest refinement or retirement when the pattern changes.

PART 06

Extensions, Integrations & Automation

ARCHITECTURE VIEW

Read-only connectors, read/write integrations, plugins, skills, durable automations, and the marketplace boundary.



LEGEND Cyan = read-only/trust controls; blue = read/write integration; purple = extension packaging; pink = skills/providers; green = durable automation. Dashed edges indicate authorization/audit observation rather than ownership.

Connectors

Read-only bridges into the user's digital world.

A connector acquires or queries data without making external changes. This is the default trust level for new services and the easiest way for Aerealith to build useful context early in onboarding.

- **Examples:** Gmail read, Google Drive read, GitHub repository/issues read, Trello boards/cards read, calendar read, Notion read, social feeds read, cloud storage read.
- **Contract:** provider auth, scopes, capability schema, fetch/list/search operations, webhook/poll support, rate limits, pagination, source IDs, freshness, and normalized event types.
- **Normalization:** preserve provider-native payload in an immutable snapshot when needed, then map important fields into Aerealith canonical records.
- **Permission inheritance:** data imported by a connector retains source visibility constraints where available.
- **Sync state:** cursor/checkpoint lives in CockroachDB; large backfills are Workflows; pages/items are queued.
- **Failure:** connector health is visible to users, and stale data is marked stale rather than presented as current.

Integrations

Read/write bridges with explicit action scopes.

An integration exposes actions: send email, create card, comment on issue, update calendar, post message, upload file, change status, etc. Integrations are implemented as typed action catalogs, not arbitrary model-generated HTTP.

- **Action schema:** name, description, input JSON Schema, required provider scopes, risk class, idempotency strategy, compensating action, preview support, audit fields.
- **Approval:** the policy layer evaluates the action against actor grants and automation scopes.
- **Execution:** fast/idempotent actions may execute directly; slow or multi-step actions enter a Queue/Workflow.
- **Result:** action result includes provider request ID, affected resource IDs, normalized status, retryability, and human-readable summary.
- **Undo:** where provider supports reversal, expose a compensating action; otherwise clearly state when an action is irreversible.
- **Rate limits:** provider-specific budget is centralized so multiple skills do not independently exhaust the same API quota.

Plugins

Installable extension bundles with a manifest and trust boundary.

Plugins are the packaging mechanism for extending Aerealith. A plugin can contribute connectors, integrations, skills, event handlers, schemas, UI cards, configuration, and documentation. It cannot bypass core identity, permission, audit, or billing systems.

- plugin_id + version + publisher
- required Aerealith API version
- connector/integration capability declarations
- OAuth providers/scopes
- actions and JSON Schemas
- events consumed/emitted
- skills included
- UI extension points
- data classifications accessed
- network destinations
- storage requirements
- permissions requested
- signature/package hash
- support/privacy URLs

FUTURE	**Advanced third-party execution should begin with declarative manifests, platform-owned adapters and reviewed APIs. Arbitrary customer code is Future scope and requires a separately accepted sandbox, quota, signing, review and incident-control architecture.**

Skills

Reasoning capabilities composed from tools.

A skill is what users experience: Project Manager, Inbox Curator, Researcher, Meeting Prep, Developer Assistant, Travel Organizer, Community Manager, Knowledge Curator. A skill can call connectors, integrations, retrieval, deterministic utilities, and other internal agents.

- **Skill manifest:** purpose, allowed tools, minimum permissions, supported inputs, expected outputs, model tier, cost class, safety/risk class, evaluation suite.
- **Composition:** skills should call stable platform capabilities rather than vendor SDKs directly.
- **Testing:** every skill gets golden-path scenarios, permission-denial tests, tool failure tests, adversarial prompt-injection tests, and cost benchmarks.
- **Versioning:** a conversation/run records which skill version produced actions.
- **User control:** skills can be disabled globally, per organization, or per user; marketplace installation does not imply write permission.

Automation Engine

Triggers + conditions + actions + policy + history.

The automation definition is declarative. Execution is event-driven and durable. A natural-language builder can create definitions, but the saved artifact is structured and inspectable.

- **Triggers:** schedule, webhook/domain event, provider event, condition watch, user command, goal milestone, connector change.
- **Conditions:** structured predicates over trusted fields; optional model classification can enrich but should not replace critical deterministic checks.
- **Actions:** typed integration actions or skill invocations.
- **Limits:** max runs, rate, spend, target resources, quiet hours, approval thresholds, expiry.
- **Notifications:** always, failures only, daily digest, silent-with-audit, or policy-defined.
- **History:** every run stores trigger evidence, evaluated conditions, policy decision, actions, results, cost, and trace ID.
- **Dry run:** preview mode evaluates what would happen without performing writes.

Marketplace

Distribution without turning trust into chaos.

The marketplace can distribute plugins, skills, connector packs, automation templates, and organization policies. Installation is separate from authorization: a user may install a plugin but still grant it zero external write permissions.

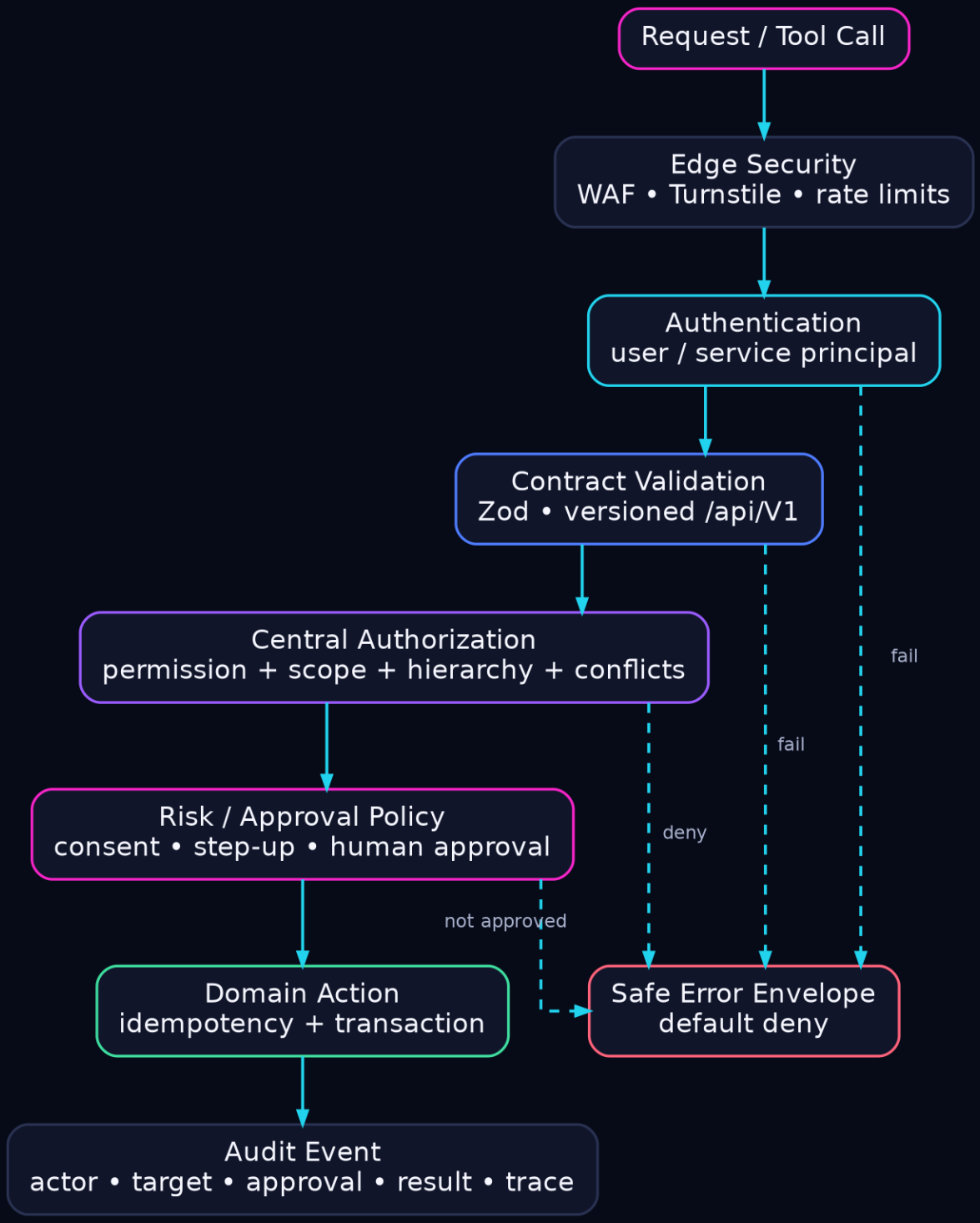
- **Publisher identity:** verified publisher records and signing keys.
- **Review tiers:** unreviewed/private, community, verified, first-party.
- **Manifest diff:** updates that request new scopes or network destinations require renewed consent.
- **Telemetry:** crash/security metrics without exposing user content to plugin authors unless explicitly permitted.
- **Kill switch:** platform can disable a malicious version while preserving data and providing migration guidance.
- **Revenue:** marketplace billing can be layered later; keep entitlement separate from capability permission.
- **Portability:** plugin APIs should be documented and versioned so ecosystem authors are not forced into Aerealith internals.

PART 07

API, Identity, Trust & Security

ARCHITECTURE VIEW

Public API contracts, events, webhooks, OAuth, secrets, tenant isolation, authorization, auditing, retention, and safe execution.



LEGEND Pink = request/risk boundary; cyan = identity/authorization; blue = validation; green = approved domain action; red = default-deny safe failure. Solid path is the successful request flow; dashed branches are failure/denial exits.

API Architecture

Everything Aerealith ships should be possible through its own APIs.

The web app, mobile clients, companion runtime, admin console, plugins, and automations should use stable platform APIs. This prevents privileged “frontend-only” behavior and forces the platform boundary to remain coherent.

- Public REST API: the canonical stable prefix is /api/V1/ (uppercase V). OpenAPI/JSON Schema, stable IDs and cursor pagination are required. Public error codes are stable lowercase dot-namespaced identifiers.
- Internal service RPC: Cloudflare service bindings where appropriate, but domain contracts remain explicit and testable. Expected failures use Result<T, E> from libs/core; transport adapters convert Result into the mandatory public response envelope.
- Webhook foundation: MVP covers signed inbound provider routes under /api/V1/webhooks/<provider>, verification, replay windows, schema validation, normalization, idempotency and processing logs. General outbound customer webhooks are Post-MVP.
- Public JSON responses use the DEC-010 success/error envelope with requestId and traceId. MCP may be exposed for tool discovery where useful, but Aerealith authorization remains authoritative.
- **SDKs:** TypeScript first; generate or hand-maintain additional SDKs after API stability.
- **Idempotency:** all externally retryable write endpoints accept an idempotency key or derive one from an execution run.

Domain Events

The connective tissue between transaction, knowledge, automation, and analytics.

Important state transitions emit versioned domain events after the transactional write commits. Events are facts about what happened, not commands telling consumers what to do.

- user.created
- organization.member.added
- connector.connected
- connector.sync.completed
- artifact.created
- document.indexed
- knowledge.fact.superseded
- goal.created
- goal.link.suggested
- automation.created
- automation.run.started
- integration.action.completed
- permission.granted
- permission.revoked
- conversation.message.created
- security.policy.denied

Use an outbox pattern or equivalent reliable publication strategy so a Cockroach transaction and its event cannot silently diverge. Consumers must be idempotent. Event payloads contain IDs and minimal safe fields; sensitive payloads are fetched through authorized APIs.

Webhook Foundation & Provider Sync

MVP Required inbound foundation: assume duplicates, reordering, missing events, retries and provider outages. Outbound customer webhook management remains Post-MVP.

46. Verify provider signature before parsing untrusted payload deeply.
47. Resolve provider account -> Aerealith connection -> tenant.
48. Persist a webhook receipt ID/hash and reject/reuse duplicates idempotently.
49. Normalize to a provider event envelope and enqueue quickly.
50. Fetch current provider state when an event is ambiguous or out of order.
51. Apply synchronization transaction with provider version/etag checks.
52. Emit Aerealith domain events after successful state change.
53. Run periodic reconciliation jobs so missed webhooks do not create permanent divergence.
54. Track provider quota, auth expiry, webhook health, and sync lag in the admin cockpit.

OAuth, Secrets & Credentials

Credentials are references with lifecycles, not strings passed around the app.

OAuth connections should store provider account IDs, granted scopes, expiry, refresh state, revocation status, and a reference to encrypted secret material. Secret values must not appear in normal application logs, model prompts, analytics payloads, or client-visible JSON.

- **OAuth:** authorization code + PKCE where applicable; state/nonce validation; exact redirect URIs; incremental scopes.
- **Secret storage:** Cloudflare Secrets for platform secrets; for large multi-tenant credential storage, use encrypted-at-application-layer fields backed by managed KMS/HSM or a dedicated secret manager as the platform matures.
- **Rotation:** provider keys and signing secrets have owner, purpose, created_at, last_used_at, rotation_due, and revocation procedure.
- **Scope minimization:** connector scopes are read-only when provider supports it; integration upgrade requests only the additional scopes required.
- **Incident:** mass revoke by provider, plugin, organization, or compromised credential class must be operable from admin tooling.

Tenant Isolation

Personal and organization data boundaries must be visible in every query path.

Aerealith is multi-tenant from the beginning. The tenant ID is part of every durable entity, search point, graph projection, event, run, cache key, and audit record where applicable. Relying only on “we remembered to add a WHERE clause” is not enough.

- **Repository layer:** centralize tenant-aware query helpers and require explicit system context for cross-tenant administrative operations.
- Qdrant: tenant, visibility, classification and lifecycle payload filters are mandatory, followed by source-level authorization re-check through @aerealith-ai/authorization.
- **Neo4j:** tenant property / database strategy selected based on scale, but application queries always bind tenant context.
- **R2:** opaque tenant namespacing and access through authorized Workers.
- **Queues/events:** tenant ID included and validated at consumer boundary.
- **Admin:** cross-tenant support access is just-in-time, audited, least-privilege, and preferably user-visible for sensitive cases.
- **Testing:** automated “two tenant” tests attempt horizontal access across every public resource type.

Security Model

Zero trust externally; explicit trust internally.

The primary threats are credential theft, horizontal tenant access, malicious plugins, prompt injection from connected content, unsafe autonomous actions, data exfiltration through model providers, webhook spoofing, excessive admin access, and supply-chain compromise.

- **Authentication:** modern passkeys/OIDC/password flow as product requires; MFA support; secure session rotation and revocation.
- Authorization: centralized @aerealith-ai/authorization service backed by normalized role, permission, hierarchy, conflict, assignment and principal-version tables. Deny by default; every protected operation requests an exact permission and scope. Authentication and authorization remain separate.
- **Prompt injection:** retrieved/connected content is untrusted data, never instructions. Tool permissions are not expanded by text inside documents or emails.
- **Egress control:** plugins and sensitive worker paths have allowlisted destinations where practical.
- **Encryption:** TLS in transit; provider encryption at rest; add application-layer encryption for selected sensitive fields.
- **Supply chain:** lockfiles, dependency scanning, signed CI artifacts, minimal build permissions, secret scanning.
- **Admin:** Cloudflare Access/Zero Trust plus Aerealith admin roles and audit. No shared root accounts.
- **Abuse:** Turnstile, rate limits, behavioral throttles, quota controls, and incident tooling.

Audit & Accountability

If Aerealith acts, it must be able to explain what happened.

Audit events are immutable append-only records of sensitive changes and actions. They are not the same as debug logs.

- audit_id
- tenant_id
- actor_type/id
- impersonation/support context
- action
- resource_type/id
- integration/provider
- automation/run_id
- permission/grant_id
- decision allow/deny
- before/after summary or change hash
- provider request/result ID
- ip/device/session where appropriate
- trace_id
- created_at

USER TRUST	**A user should be able to ask: “What did you do to my Trello board yesterday?” and Aerealith should answer from audit/action history, not hallucinate from conversation memory.**

Retention differs by plan, organization policy, compliance need, and event sensitivity. Debug payloads can expire quickly; security/action audit may need much longer retention. Avoid storing secrets or unnecessary message bodies in audit events.

Deletion, Retention & Data Export

Deletion is a distributed workflow.

Because Aerealith derives vectors, graph relations, summaries, caches, and backups from source data, “delete row” is insufficient. Deletion is a versioned Workflow that locates and removes or tombstones every derivative according to retention and legal policy.

55. Mark deletion requested and freeze new derivative generation.
56. Check legal hold, organization retention, billing disputes, and required audit retention.
57. Revoke external connector/integration credentials if account deletion requires it.
58. Delete or tombstone Cockroach entities and permission grants.
59. Delete R2 artifacts and derivative extraction objects.
60. Delete Qdrant vectors by stable source/provenance IDs as part of the deletion manifest; derived indexes must never outlive authoritative source deletion policy.
61. Delete/rebuild Neo4j nodes/edges derived solely from deleted content.
62. Evict KV/semantic caches.
63. Schedule backup expiry according to documented recovery window.
64. Produce deletion completion record with exceptions.

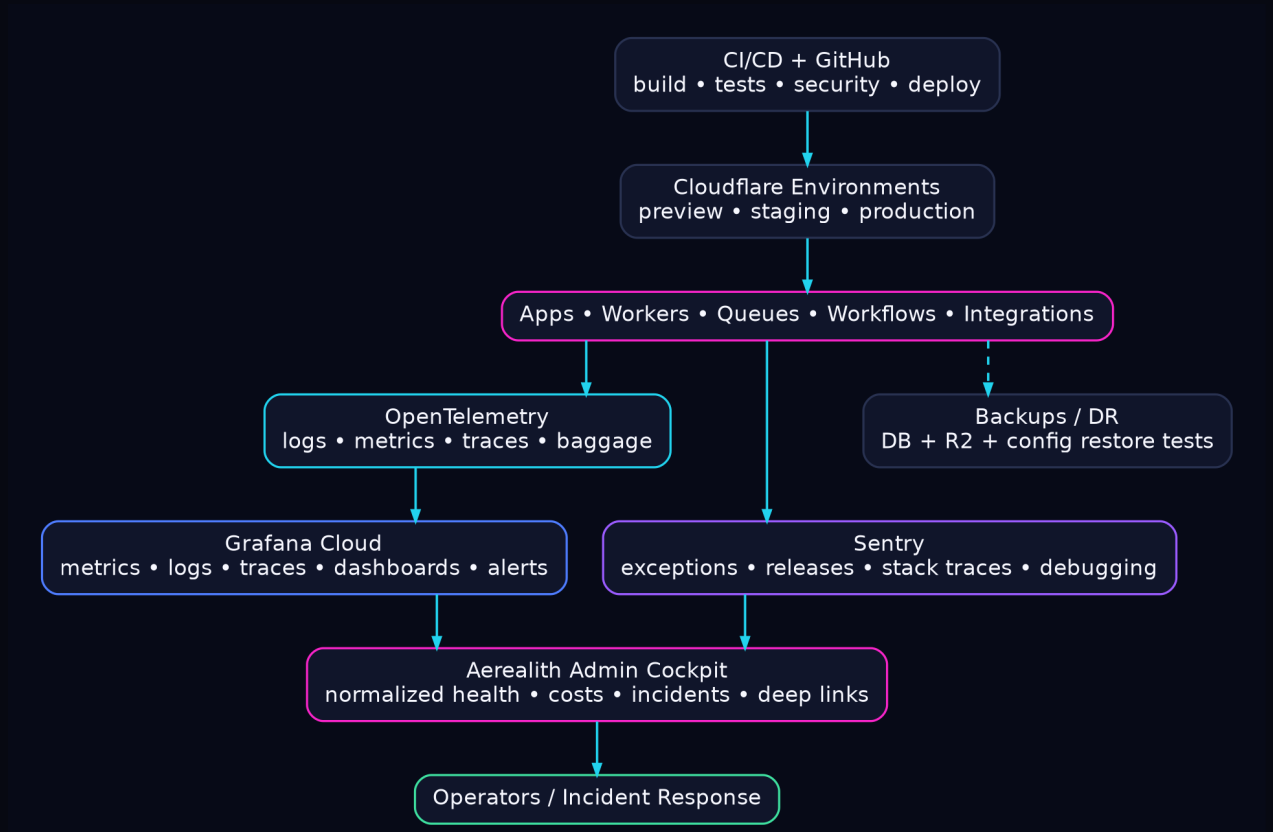
Data export mirrors the same provenance map: transactional JSON/CSV, user-visible artifacts, conversations, goals, automations, and optionally a portable knowledge export. Portability builds trust and reduces vendor lock-in.

PART 08

Operations, Reliability & Delivery

ARCHITECTURE VIEW

Telemetry, application debugging, admin operations, email, billing, SLOs, disaster recovery, environments, testing, portability, and cost control.



LEGEND Pink = production services/admin surface; cyan = OpenTelemetry; purple = debugging; blue = observability platform/deployment; green = operators. Solid arrows are telemetry/deployment paths; dashed arrows indicate backup/secondary evidence paths.

Observability, Reliability & Incident Architecture

OpenTelemetry is the instrumentation layer; vendors are destinations.

Instrument Aerealith with OpenTelemetry as the vendor-neutral telemetry contract. Trace context must propagate across Cloudflare Workers, service bindings, Queues, Workflows, Durable Objects, database operations, provider calls, Discord events, webhooks, AI calls, Qdrant retrieval, Neo4j traversal, Resend delivery and user-visible actions where technically supported. Telemetry is correlated by requestId, traceId, user/organization-safe identifiers, deployment, service and release.

- **Metrics:** latency, error rate, queue depth, workflow duration, sync lag, provider quota, model cost/tokens, cache hit, retrieval quality, graph/vector lag.
- **Traces:** user request -> model/router -> retrieval -> tools -> external provider -> response.
- **Logs:** structured, low-cardinality labels, redaction, trace IDs, tenant-safe metadata.
- **Events:** product/business analytics remain separate from operational logs; do not ship private content by accident.
- **SLOs:** measure user outcomes such as successful action completion and fresh connector sync, not only HTTP 200 rates.

CLOUD	**Grafana Cloud - primary telemetry aggregation and dashboards** official
SELF-HOST	Grafana OSS + Prometheus + Loki + Tempo, managed with Grafana Alloy/OpenTelemetry Collector. project
WHY	Grafana Cloud supports OTLP ingestion for metrics, logs and traces, making OTel the portability layer.

Sentry - Application Error & Developer Debugging Plane

Developer-centric error and issue debugging.

Sentry owns actionable application errors: frontend/backend exceptions, stack traces, releases, source maps, regression detection, performance spans where useful, suspect commits and developer issue workflow. Sentry is not the authoritative metrics/log archive. Every Sentry issue/event should carry requestId/traceId, service, environment, release and safe tenant context so the Aerealith admin cockpit can deep-link directly to the developer failure.

CLOUD	**Sentry - error monitoring and developer debugging** official
SELF-HOST	Self-hosted Sentry is available but operationally heavier; GlitchTip is a lighter Sentry-compatible alternative for some use cases. project
WHY	Avoid duplicating all telemetry in every vendor. Sentry is the “why did this code fail?” surface; Grafana is the platform-wide operational view.

- **Tag safely:** service, environment, release, route, run type; avoid raw user identifiers and high-cardinality secrets.
- **Source maps:** upload from CI with release identifiers.
- **Alerts:** page only on actionable production regressions; lower-severity noise becomes triage backlog.
- Correlation: the admin cockpit surfaces normalized incident context and deep links into Grafana Explore/dashboards and Sentry issues rather than recreating either forensic UI.

Grafana Cloud - Primary Operations & Telemetry Plane

Do not pay three times for the same APM problem.

Grafana Cloud is the primary operational source of truth for service health, metrics, logs, traces, SLOs, alerting, capacity, queue depth, workflow latency, database health, Qdrant/Neo4j health, provider health, AI usage/cost and business-operational telemetry. Grafana Faro supplies browser telemetry. OpenTelemetry/Grafana Alloy is the preferred collection/export boundary. Datadog may remain only for a distinct capability with a named consumer; do not duplicate every metric/log/trace into multiple APM vendors by default.

- Dashboards: platform overview, service RED/USE metrics, AI cost/latency, integration health, queue/workflow backlog, relational DB, Qdrant, Neo4j, email delivery, security events, billing, deployment health and user-impact indicators.
- Alerting: define SLOs and error budgets for critical user journeys; alerts must be actionable, routed by ownership, deduplicated and linked to runbooks. Page on symptoms/user impact rather than raw infrastructure noise whenever possible.
- **Instrument once:** OpenTelemetry reduces the switching cost. Route data at collector/gateway level rather than embedding vendor SDK logic everywhere.
- **Cost control:** maintain a telemetry budget per environment; development and preview should sample aggressively and expire logs quickly.

RECOMMENDATION	**Grafana Cloud + Sentry is the default Aerealith observability pair. Treat Datadog as optional/special-purpose, not a third mandatory APM.**

Admin Cockpit

Aerealith should interpret its own platform, not merely embed twelve vendor dashboards.

The admin console aggregates normalized operational data through Aerealith backend APIs. It should show service health, user-impacting incidents, AI costs, provider health, queue/workflow status, connector sync lag, knowledge indexing lag, storage growth, security events, billing, feature rollouts, and deployment state.

- **Overview:** current incidents, SLO status, active users, failed runs, queue age, monthly spend trajectory.
- **AI Ops:** cost/token/latency by model/provider/skill; cache rate; fallback rate; model errors.
- Knowledge Ops: ingestion backlog, failed extraction, Qdrant collection size/query latency/index health, Neo4j projection lag, stale sources, rebuild status and embedding-generation drift.
- **Integration Ops:** OAuth expiry, provider API error/rate-limit trends, webhook lag.
- **Security:** denied actions, suspicious auth, admin access, secret rotation, plugin incidents.
- Drill-down: deep link to Grafana Explore/dashboard and Sentry issue using shared traceId/requestId/release metadata. Preserve the admin dashboard as the cross-provider control plane, not a replacement for specialist forensic tools.
- **Narrative:** an ops skill can summarize “what changed” and suggest likely causes, but raw metrics and links remain available.

Resend - Transactional Email Provider

Transactional email, notifications, and platform mail.

Resend is the managed outbound transactional email provider behind an internal email/notification adapter. Use it for account verification, password/security notices, organization invitations, approval requests, ticket notifications, integration/workflow alerts and user-authorized automation email. Domain authentication (SPF/DKIM/DMARC as applicable), per-environment API keys, least privilege, idempotency keys, suppression/bounce/complaint handling, webhook verification, template/version ownership, rate limiting and auditable user-triggered sends are architecture requirements. Business logic must never depend directly on Resend response types so another delivery provider or self-hosted mail system can replace it.

CLOUD	**Resend - developer email API** official
SELF-HOST	Postal is a self-hosted mail delivery platform; for most teams, managed delivery is preferable because reputation and deliverability are operational specialties. project
WHY	Use separate API keys by environment/purpose, least privilege, domain authentication, webhook verification, suppression handling, and audit of user-triggered sends.

- **Queue outbound mail:** user-facing requests should not block on email provider latency.
- **Idempotency:** dedupe security/invite/automation notifications by event ID.
- **Content:** never send secrets; sensitive notifications link back to authenticated Aerealith.
- **Inbound:** if receiving email later, treat inbound bodies/attachments as untrusted connector content subject to prompt-injection protections.

Billing & Entitlements

Stripe or equivalent should be a billing system, not the authorization system.

Aerealith will eventually need plans, metering, marketplace purchases, organization billing, quotas, overage policy, and possibly usage credits. Keep an internal entitlement model in CockroachDB so plan capabilities are stable even if billing provider changes.

CLOUD	**Stripe - recommended managed billing/payment provider** official
SELF-HOST	Kill Bill for a self-hosted billing platform, plus a payment processor; self-hosting card processing itself is not recommended. project
WHY	Store provider customer/subscription IDs, but derive allowed Aerealith capabilities from an internal entitlement snapshot updated by verified provider events.

- **Meter:** AI cost, storage, premium connector usage, automation runs, plugin entitlements as product requires.
- **Do not meter blindly:** avoid pricing that makes users afraid to talk to the companion. Bundle normal conversation and expose limits clearly.
- **Webhooks:** billing state changes only from verified events/reconciliation, not client claims.
- **Failure:** grace periods and read-only access are better than destructive lockout for transient payment failures.

Reliability & SLOs

Design around user-visible outcomes.

SLO	Initial target	Meaning
Core conversation availability	99.9% initial target	User can authenticate, send/receive companion messages, and access existing data.
Safe action success	99.5%+ excluding provider outages	Approved actions execute exactly once or present a recoverable failure.
Connector freshness	provider-specific	High-priority connected sources stay within a stated sync lag.
Knowledge indexing	P95 target by file size	New authorized content becomes retrievable within an expected window.
Audit completeness	effectively 100% for sensitive actions	No privileged/write action without audit record or compensating incident.

Error budgets should influence feature rollout. If action execution or connector sync is unhealthy, Flagship can disable high-risk features or switch to read-only/degraded behavior while preserving conversation and visibility.

Disaster Recovery

Derived stores are rebuildable; authority and source artifacts are recoverable.

- **CockroachDB:** managed backups/PITR according to plan; regularly test restore into an isolated environment.
- **R2:** object versioning/lifecycle where appropriate; maintain backup/export strategy for critical platform-owned artifacts.
- Qdrant: keep deterministic rebuild manifests from authorized chunks + embedding generation; managed clusters may use snapshots/backups, but the index remains reconstructable derived state.
- **Neo4j:** back up graph and keep deterministic projection/rebuild paths from source entities and knowledge events.
- **Secrets:** document recovery and rotation; backups do not contain plaintext secrets outside approved systems.
- **Infrastructure:** Wrangler/IaC, DNS/config, queues, bindings, feature flags, dashboards, alerting definitions are version-controlled where possible.
- **Exercises:** quarterly restore drill as the platform matures: restore DB, recover artifacts, rebuild one vector collection and graph projection, validate permissions, then run smoke tests.

RPO/RTO	**Set formal Recovery Point and Recovery Time Objectives before paid production launch. “We have backups” is not an RTO.**

Deployment & Environments

Dev, preview, staging, and production are separate trust domains.

Use environment-specific Cloudflare resources, relational database branches/clusters as appropriate, isolated Qdrant and Neo4j environments, Resend keys/domains, model budgets, Sentry projects/releases and Grafana Cloud labels/stacks. Production credentials never enter preview builds.

- **Local:** Wrangler emulation + local/test DB + mocked providers or dedicated sandbox accounts.
- **Preview:** per-PR frontend/Workers where practical; safe seeded tenant; external writes disabled or pointed at test resources.
- **Staging:** production-like integration tests with dedicated provider accounts and representative data volumes.
- **Production:** protected deploys, change log, migrations, canary/Flagship rollout, rollback plan.
- **CI gates:** lint, typecheck, unit, integration, tenant isolation, permission matrix, migration validation, security scan, contract tests, cost smoke tests.
- **Release evidence:** record Git commit, worker deployment IDs, schema version, plugin API version, and feature-flag state in release metadata.

Testing Strategy

The hardest bugs live between services, permissions, and retries.

- **Unit:** domain rules, permission evaluation, normalizers, parsers, cost calculations, action schemas.
- **Contract:** provider adapters against recorded/sandbox responses; plugin/skill manifests against JSON Schema.
- Integration: relational transactions + transactional outbox/event publication + Queue/Workflow processing + Qdrant/Neo4j projection.
- **Two-tenant security:** systematically try to access every resource from the wrong tenant.
- **Replay:** deliver duplicate and out-of-order webhooks/queue messages.
- **AI evaluations:** groundedness, correct tool selection, refusal to obey prompt injection in sources, cost, latency, interruption handling.
- **Chaos:** provider timeout, rate limit, expired token, queue retry, workflow crash, graph/vector unavailable.
- **Migration:** read old/write new, dual-write reconciliation where needed, backfill verification, rollback safety.
- **Production synthetic:** low-risk canary flows that verify auth, conversation, retrieval, and a sandbox integration.

Self-Hosted Escape Hatch Map

Cloudflare-first should still leave Aerealith with choices.

Managed default	Self-host / portable alternative
Workers	Kubernetes + Hono/Fastify + Envoy/NGINX
R2	MinIO / Ceph RGW
KV	Valkey / Redis
Queues	NATS JetStream / RabbitMQ
Workflows	Temporal
Durable Objects	Orleans/Akka or NATS + transactional state
Flagship	Unleash / other OpenFeature provider
Qdrant Cloud	Qdrant self-hosted (Docker/Kubernetes)
AI Gateway	LiteLLM Proxy
Workers AI	vLLM / TGI / Ollama
Turnstile	mCaptcha / ALTCHA
Zero Trust Access	Authentik / Keycloak + reverse proxy
CockroachDB Cloud	self-host CockroachDB or PostgreSQL portability target
Neo4j AuraDB	self-managed Neo4j / Memgraph
Grafana Cloud	Grafana OSS + Prometheus/Loki/Tempo
Sentry Cloud	self-host Sentry / GlitchTip
Resend	Postal (with significant deliverability ops)
Stripe	Kill Bill for billing logic + external payment processor

The point is not to run two stacks. The point is to keep domain logic, schemas, event contracts, and observability portable enough that a vendor change is a migration project rather than a rewrite.

Cost Architecture

Put a price tag on data movement and background intelligence.

Aerealith can become expensive because it combines storage, AI inference, indexing, external API calls, observability, and continuous background work. The cost model must exist at the architecture layer.

- **Cost dimensions:** per active user, per tenant, per GB source, per million chunks/vectors, per automation run, per provider call, per model call, per GB logs.
- **Background budget:** each tenant gets a background intelligence budget so crawler/recommendation loops cannot run indefinitely.
- **Freshness tiers:** revalidate high-value volatile knowledge frequently; static documents need no repeated model work.
- **Observability retention:** short retention for verbose logs; long retention for low-volume security/audit records.
- **Storage lifecycle:** use R2 Standard for active objects and move only genuinely cold objects to infrequent access after modeling retrieval charges and minimum storage duration.
- **Model budget:** cache and route by task; large models are an escalation, not a default.
- **Third-party overlap:** avoid duplicate APM/log ingestion across Grafana, Sentry, and Datadog.
- **Chargeback:** enterprise admin can see where usage originates: skill, plugin, connector, automation, model, or project.

Operational Data Model

Runs are the spine of system-wide traceability.

Use a common run model across AI requests, ingestion, sync, skills, automations, workflows, actions, exports, and deletions. A run is the unit users/admins can inspect when something succeeds, fails, costs money, or changes data.

- run_id
- run_type
- tenant_id
- initiator actor/automation/event
- parent_run_id
- conversation_id
- goal_id
- status
- started_at/ended_at
- trace_id
- input_ref
- output_ref
- policy_decision_id
- cost summary
- retry_count
- error class
- provider request IDs
- artifacts produced
- actions performed

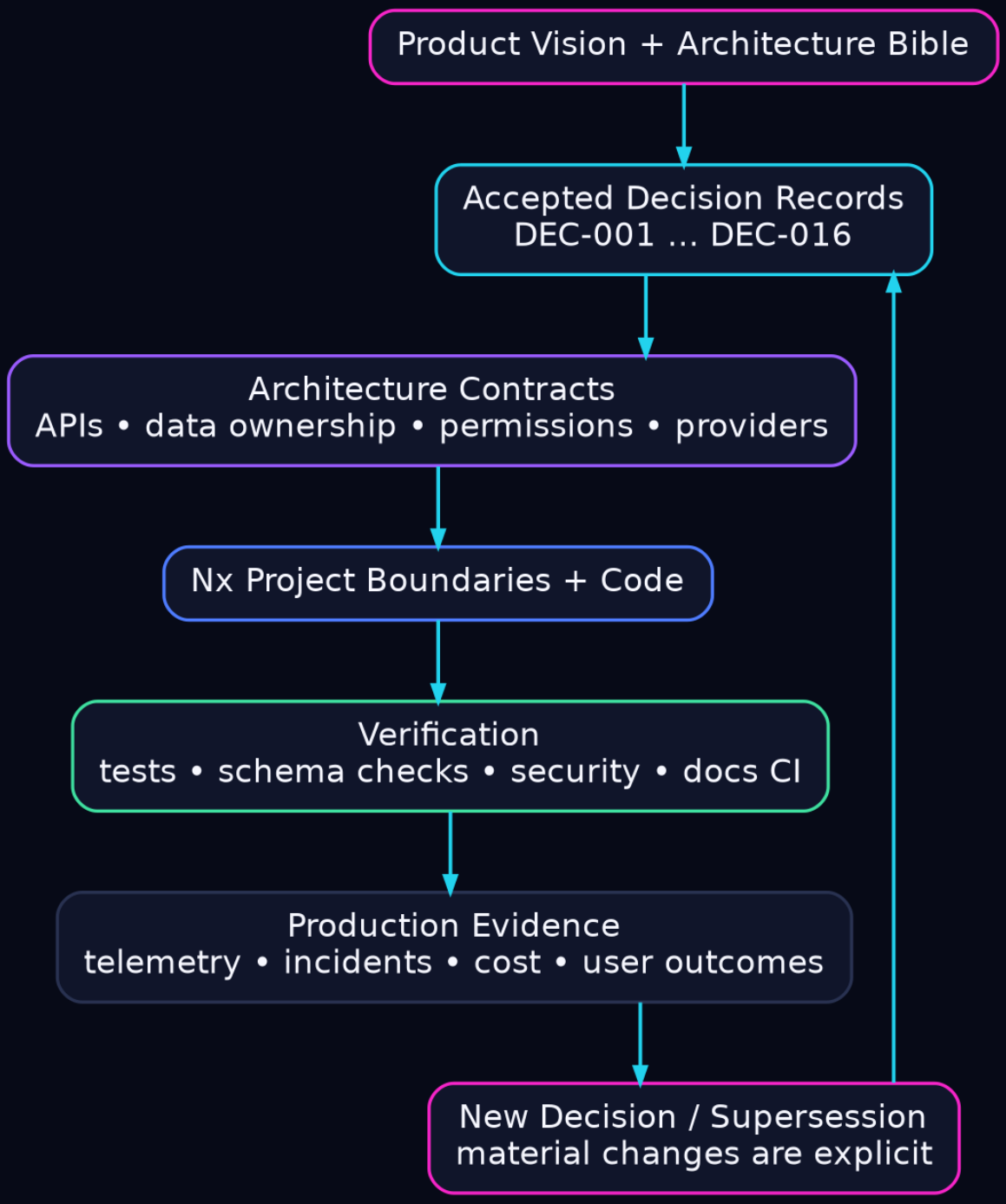
Parent/child runs create a tree: a conversation turn starts a Research skill run; that starts retrieval and two provider queries; the user approves an action; an integration run updates Trello; an audit event records the mutation. One trace/run tree lets Aerealith explain the whole chain.

PART 09

Governance & Architecture Reference

ARCHITECTURE VIEW

Operator access, prompt-injection defense, data classification, roadmap gates, binding decisions, service catalog, implementation checklist, and references.



LEGEND Pink = product vision/material change; cyan = binding decisions; purple = architecture contracts; blue = implementation; green = verification. The loop means production evidence can trigger an explicit superseding decision - never a silent architectural rewrite.

Admin Roles & Support Access

Support must not become an invisible superuser backdoor.

Platform Owner: full emergency access; protected by strongest MFA, just-in-time use, and audit.

Security Admin: security policies, secrets metadata, incidents, access review; no routine user-content browsing.

Operations Admin: service health, runs, queues, provider status, non-content telemetry.

Support: tenant-scoped diagnostic access, preferably metadata first; content access only with reason/approval.

Billing Admin: plans, invoices, entitlements; no knowledge content.

Developer: logs/errors/preview; production access limited and audited.

RULE	**Impersonation/support sessions carry a reason, ticket/reference, expiry, actor identity, and visible audit record. Avoid permanent “god mode.”**

Prompt-Injection Defense

Connected data is hostile until proven otherwise.

Aerealith will intentionally read email, web pages, documents, issue comments, social content, and plugin output. Any of these can contain text telling the model to ignore its rules, reveal secrets, or execute tools. That text is data, not authority.

- **Instruction hierarchy:** system/policy/tool contracts are separated from retrieved content.
- **Tool firewall:** model proposes typed actions; policy engine independently evaluates them.
- **Least privilege:** retrieval skill does not automatically possess write integrations.
- **Source labeling:** context chunks are marked as untrusted source data with provenance.
- **Secret isolation:** API keys and refresh tokens never enter model context.
- **High-risk confirmation:** sensitive write/destructive/financial actions use deterministic approval rules regardless of model confidence.
- **Plugin review:** plugin tool descriptions and outputs are also untrusted unless first-party signed and still subject to permission checks.
- **Evaluation:** maintain a living corpus of injection attacks drawn from real connector types and run it in CI.

Data Classification

The same pipeline should not treat every byte equally.

Class	Examples/intent	Controls
Public	intended for public use	normal retrieval; may enter public corpus only by explicit policy.
Internal	normal tenant data	tenant-authenticated processing and approved providers.
Confidential	sensitive business/personal content	restricted tools/providers, stronger logging redaction, shorter derived caches.
Restricted	secrets, highly sensitive categories, regulated data where applicable	minimal access, application-layer encryption, provider allowlist or local processing, no general-purpose logs.

Classification can be user-set, organization-policy-set, source-inherited, or machine-suggested. A machine suggestion can raise sensitivity automatically but should not silently lower a user/organization classification.

Roadmap: Architecture Before Scale

Build the bones in the order that reduces rework.

MVP Required - Contracts & Core Data: canonical vocabulary; /api/V1/; Result and namespaced error codes; mandatory API envelope; stable IDs; normalized domain/persistence mapping; PostgreSQL + Drizzle baseline; CockroachDB compatibility tests; centralized authorization.

MVP Required - Discord Flagship: Discord is the primary MVP persona and first flagship integration. Deliver the eleven required Discord modules, module registry/lifecycle, permissions, moderation, tickets, transcripts, logging/audit, welcome/roles and activity summaries. Command Manager and Discord Health are MVP Should-Have.

MVP Required - Assistant Surface: suggest, summarize, explain and prepare action proposals. No autonomous meaningful execution, durable personal/community memory, advanced model routing or automatic punishment. All actions pass deterministic authorization, risk, approval and audit controls.

MVP Required - Integration Foundation: product-integration lifecycle, OAuth/secret handling, inbound webhook foundation, provider normalization, health/disconnect/revocation, basic notification preferences and deterministic ticket summaries.

Post-MVP - 1.3 Knowledge & Companion Memory: durable memory, Knowledge Garden ingestion, Qdrant semantic retrieval, Neo4j graph projections, provenance/freshness, goal awareness, public/private contribution controls and advanced model routing.

Post-MVP - 1.3 Autonomy: learned-pattern recommendations, user-granted automations, proactive behaviors, richer goal/relationship awareness, compensation/undo and long-running companion continuity.

Future - Ecosystem: plugin/skill SDK, marketplace, organization policies, billing/entitlements, enterprise controls, richer connector catalog and customer-extensibility only through reviewed platform APIs/sandbox models.

Future - Scale: regional/data-residency decisions, dedicated clusters where justified, advanced cost routing, formal SLO/error budgets, disaster-recovery exercises and multi-region resilience.

Accepted Decision Register

DEC-001 through DEC-016 are binding architecture constraints for this edition.

DEC	Binding decision	Architecture consequence
DEC-001	Canonical public API prefix is /api/V1/.	All stable public REST, docs, tests and webhooks use the exact uppercase-V form.
DEC-002	Discord MVP = 11 required modules + 2 should-have; registry foundation mandatory.	MVP acceptance is Discord-operator-first and module lifecycle/permissions are testable.
DEC-003	MVP assistant suggests/explains; no autonomous meaningful execution, durable memory or advanced routing.	AI remains non-authoritative; autonomy and durable memory are Post-MVP.
DEC-004	PostgreSQL default + Drizzle; CockroachDB is a compatibility target.	DB abstractions and CI must prove Cockroach-specific safety before distributed deployment.
DEC-005	Canonical shared libraries with narrow responsibilities.	core/contracts/db/api/ui/content/flags/observability/utils boundaries are enforced by Nx.
DEC-006	Persistence-domain mapping is owned by data layer.	Domain and contracts never depend on ORM/persistence schemas.
DEC-007	Error codes are immutable const-object values with namespaced strings.	Public errors are stable identifiers; human messages remain mutable.
DEC-008	Result<T,E> lives in libs/core.	Expected failures cross domain/data boundaries without transport coupling.
DEC-009	Persistence records, domain entities and external contracts are separate forms.	Repositories map records and return domain entities/Result, never raw rows.
DEC-010	Every stable public JSON response uses the mandatory envelope.	requestId/traceId correlation is always present; sensitive internals are redacted.
DEC-011	Platform providers are distinct from product integrations.	Cloudflare/Grafana/Sentry are operator infrastructure; Discord/user accounts have consent lifecycles.
DEC-012	MVP webhook foundation is secure inbound receipt/verification/normalization/idempotency/logging.	Outbound customer webhook management is deferred.
DEC-013	Basic notification preferences + deterministic staff ticket summaries are MVP.	AI summaries are optional and must degrade safely when AI is disabled.
DEC-014	Discord community operators are primary MVP personas.	Individual digital-life foundations remain, but do not outrank Discord end-to-end MVP work.
DEC-015	Every capability has one unambiguous scope classification.	Use MVP Required, MVP Should-Have, Post-MVP - <release>, Future, Parking Lot, Out of Scope or Rejected.
DEC-016	Authorization is centralized and normalized in	users.role is deprecated; normalized

	@aerealith-ai/authorization.	assignments + principal versions are authoritative; default deny.

The Bible describes the long-term architecture, but release scope must follow the accepted decision records. Where future architecture exceeds MVP boundaries, this document classifies it explicitly as Post-MVP or Future rather than treating it as launch scope.

Service Catalog

Managed default, role, and portability target.

Managed	Role	Portable alternative
Cloudflare Workers	edge API/orchestration	Kubernetes + Hono/Fastify
R2	object storage	MinIO/Ceph
KV	read-heavy edge cache/config	Valkey
Queues	async work	NATS/RabbitMQ
Workflows	durable orchestration	Temporal
Durable Objects	coordination/live state	actor/state service
Flagship	feature flags	Unleash/OpenFeature
Qdrant Cloud	semantic/vector retrieval	Qdrant self-hosted
AI Gateway	AI traffic control	LiteLLM
Workers AI	managed inference	vLLM
CockroachDB	transactional SQL	self-host CRDB/PostgreSQL
Neo4j AuraDB	graph traversal	self-host Neo4j/Memgraph
Resend	transactional email	Postal
Grafana Cloud	metrics/logs/traces	Grafana OSS stack
Sentry	error debugging	self-host/GlitchTip
Stripe	billing/payments	Kill Bill + processor

Build Checklist

What must exist before Aerealith can safely become proactive.

- [ ] Canonical entity IDs and tenant envelope implemented in shared database library.
- [ ] Central permission/capability evaluator used by every write action.
- [ ] Artifact metadata + R2 upload/download broker with hash, retention and access checks.
- [ ] Domain event envelope + reliable publication/outbox.
- [ ] Run model and trace propagation through Workers, Queues and Workflows.
- [ ] OpenTelemetry export to Grafana Cloud; Sentry releases/source maps wired in CI.
- [ ] AI Gateway + model registry/router with per-run cost accounting and caching.
- [ ] Knowledge ingestion Workflow with idempotent extraction/chunk/embed/project steps.
- [ ] Qdrant tenant/visibility/classification filters, provenance metadata, source authorization re-check, embedding versioning, deletion manifests, backups/rebuilds and load tests.
- [ ] Neo4j projection/rebuild process and minimal ontology.
- [ ] Connector SDK with read-only scope declaration, sync cursors and provider health.
- [ ] Integration action SDK with JSON Schema, risk, idempotency, approval and compensation.
- [ ] Automation schema with trigger/conditions/actions/limits/notification/history.
- [ ] User-facing permission center, action history, memory viewer and data export/delete flows.
- [ ] Admin cockpit with incidents, AI cost, queue/workflow health, indexing lag and provider state.
- [ ] Two-tenant security tests, webhook replay tests and prompt-injection evaluation suite.
- [ ] Feature-flag strategy for staged rollout of risky capabilities.
- [ ] Disaster-recovery restore test and vector/graph rebuild rehearsal before paid GA.

Reference Links

Official managed-service documentation and key standards.

Aerealith AI https://aerealith.com/

Cloudflare Workers https://developers.cloudflare.com/workers/

Cloudflare R2 https://developers.cloudflare.com/r2/

Cloudflare KV https://developers.cloudflare.com/kv/

Cloudflare Queues https://developers.cloudflare.com/queues/

Cloudflare Workflows https://developers.cloudflare.com/workflows/

Cloudflare Durable Objects https://developers.cloudflare.com/durable-objects/

Cloudflare Flagship https://developers.cloudflare.com/flagship/

Qdrant Cloud https://qdrant.tech/cloud/

Cloudflare AI Gateway https://developers.cloudflare.com/ai-gateway/

Cloudflare Workers AI https://developers.cloudflare.com/workers-ai/

Cloudflare Turnstile https://developers.cloudflare.com/turnstile/

Cloudflare Zero Trust https://developers.cloudflare.com/cloudflare-one/

CockroachDB Docs https://www.cockroachlabs.com/docs/

Drizzle ORM https://orm.drizzle.team/

Qdrant self-hosting https://qdrant.tech/documentation/guides/installation/

Neo4j AuraDB https://neo4j.com/docs/aura/

Resend https://resend.com/docs/introduction

Grafana Cloud https://grafana.com/docs/grafana-cloud/

Sentry https://docs.sentry.io/

OpenTelemetry https://opentelemetry.io/docs/

OpenFeature https://openfeature.dev/

Temporal https://docs.temporal.io/

NATS https://docs.nats.io/

Valkey https://valkey.io/

MinIO https://min.io/

Unleash https://docs.getunleash.io/

LiteLLM https://docs.litellm.ai/

vLLM https://docs.vllm.ai/

Vendor pricing and limits change. Treat this Bible as architecture guidance, and verify current service limits/pricing before capacity commitments or contractual decisions.

Closing Principle

Aerealith should make complexity disappear without hiding control.

The architecture succeeds when the user experiences one companion while the platform underneath remains rigorously separated: identity from inference, authority from suggestion, source truth from derived indexes, observation from action, installation from permission, and managed convenience from portability.

The hardest requirement is not connecting every service. It is connecting them while preserving understandable control. Aerealith should learn enough to be helpful, ask enough to be trusted, automate enough to remove drudgery, and record enough to explain itself.

DESIGN TEST	**Before adding any subsystem, ask: Does this reduce user friction? Does it preserve user agency? Can we explain and undo what it does? Is the source of truth clear? Can we observe and recover it? Can we afford it at scale?**

If the answer is no, the feature is not ready - no matter how futuristic it looks.

AEREALITH AI

Your Digital Life, Intelligently Connected

ARCHITECTURE COMPLETENESS MATRIX

What must exist for Aerealith to be architecturally complete

This matrix is a design gate: a feature is not architecturally complete until ownership, trust boundaries, failure behavior, observability, data lifecycle and portability are defined.

Architecture domain	Required definition / gate
Domain model & vocabulary	Canonical entities, stable IDs, bounded contexts, provider/product terminology, lifecycle/state machines, ownership and invariants.
Identity & authentication	Users, service principals, sessions, MFA/passkeys where adopted, OAuth/OIDC boundaries, token/session rotation, recovery and compromise handling.
Authorization	DEC-016 centralized normalized permissions, scopes, hierarchy/conflicts, principal versions, default deny, admin-rank rules, revocation and cache invalidation.
Multi-tenancy	Organization/community/user ownership, tenant-context propagation, isolation in SQL/Qdrant/Neo4j/R2, quotas, billing ownership and cross-tenant test coverage.
API & contracts	/api/V1/, Zod/provider-neutral contracts, mandatory response envelope, stable error codes, request/trace IDs, idempotency, pagination and versioning/deprecation policy.
Events & async processing	Canonical event envelope, transactional outbox, Queues, Workflows, retry/backoff, idempotent consumers, DLQ/quarantine, ordering assumptions and replay policy.
Webhooks	Raw-body verification, signatures/timestamps, replay prevention, validation, normalization, deduplication, secret rotation, disconnect behavior and safe logs.
Storage & data lifecycle	Cockroach/Postgres transactional truth, R2 blobs, Qdrant semantic index, Neo4j graph projection, metadata/provenance, retention, legal/user deletion and rebuild manifests.
Knowledge Garden	Ingestion/extraction/chunking, provenance, public/private contribution, source freshness, confidence, versioning, retrieval policy, revalidation and deletion propagation.
AI architecture	Provider abstraction, AI Gateway, model budgets, prompt/context policy, structured outputs, tool capability scopes, safety checks, fallbacks, cost accounting and non-AI degradation.
Companion memory & goals	Explicit memory classes, consent, user-visible controls, temporal validity, goals/relationships, pattern learning, correction/forgetting and Post-MVP autonomy boundaries.
Connectors & integrations	Read-only connector vs read/write integration semantics, OAuth scopes, health, reconnect, revoke/delete, rate limits, provider adapters and

	permission-aware actions.
Modules, plugins & skills	Manifest/registry contracts, lifecycle, dependencies, configuration schemas, capability declarations, signing/review, sandboxing rules and version compatibility.
Automation engine	Triggers, conditions, actions, approval policy, delegated permission, retries, compensation/undo, schedules, concurrency, human override, audit and kill switch.
Discord platform	DEC-002 module scope, shared client/rate limiting, shard strategy when needed, permissions/role hierarchy, moderation/tickets/logging and provider health.
Email & notifications	Internal notification service, Resend adapter, channel preferences, critical-notice policy, templates, delivery/webhook state, suppression and deterministic fallbacks.
Frontend & onboarding	React/Vite/router/query architecture, accessibility, feature flags, avatar/conversation onboarding, interruptible voice/chat UX, permission education and failure recovery.
Admin/control plane	Users/orgs/roles/permissions/sessions/audit/system health, provider health, AI ops, knowledge ops, billing, deployment status, incident summaries and safe operator actions.
Observability	OpenTelemetry, Grafana Cloud primary telemetry, Sentry errors, Faro browser signals, structured Pino logs, shared correlation, SLOs/error budgets, alerts and runbooks.
Security	Threat model, secrets/KMS, encryption in transit/at rest, SSRF/egress controls, prompt-injection/tool isolation, CSP/WAF/rate limits, supply-chain security and incident response.
Privacy & governance	Consent, data minimization, training/public-corpus opt-in, data classification, retention, export/delete, auditability, regional/residency policy and compliance roadmap.
Reliability & DR	Backups/restore tests, RPO/RTO, provider outage behavior, queue replay, database failover, Qdrant/Neo4j rebuild, config/IaC recovery and dependency degradation plans.
Performance & scale	Capacity budgets, cache strategy, hot-path latency, connection pooling, queue backpressure, Qdrant/Neo4j query budgets, sharding/partition triggers and load tests.
Cost architecture	Per-user/org usage accounting, AI/token cost, storage/index/telemetry retention, provider quotas, budget alerts, free/pro entitlements and cost-aware routing.
Deployment & environments	Dev/preview/staging/prod isolation, Cloudflare bindings, secret separation, migrations, canary/rollback, feature flags, Docker portability and infrastructure-as-code.
Testing & release gates	Unit/integration/E2E/security/a11y/visual/contract/migration/load/chaos where useful; DEC-015 scope classifications; objective release exit criteria.
Portability & self-hosting	Provider adapters, Dockerfiles, standard protocols, export formats, replacement map for Cloudflare/DB/Qdrant/Neo4j/Grafana/Sentry/Resend and migration runbooks.
Architecture governance	Decision records, supersession policy, dependency boundaries, threat/design reviews, service ownership, versioned diagrams,

	architecture drift checks and documentation CI.

REFERENCE SERVICE BOUNDARIES

Managed defaults and self-hostable paths

Capability	Managed/default	Self-host/portable	Authority
Transactional SQL	PostgreSQL/CockroachDB hosting	PostgreSQL or self-hosted CockroachDB	Authoritative entities + authorization
Object/blob storage	Cloudflare R2	MinIO/Ceph RGW	Authoritative source artifacts
Vector retrieval	Qdrant Cloud	Qdrant Docker/Kubernetes	Derived semantic index
Knowledge graph	Neo4j AuraDB	Neo4j self-managed / Memgraph alternative	Derived relationship projection
Email	Resend	Postal or alternate SMTP/API provider	Delivery provider only
Metrics/logs/traces	Grafana Cloud	Grafana OSS + Prometheus/Loki/Tempo	Operational telemetry
Application errors	Sentry Cloud	Self-hosted Sentry / GlitchTip	Developer debugging projection
AI traffic	Cloudflare AI Gateway	LiteLLM / custom gateway	Routing/cost/observability boundary
Managed inference	Workers AI + external providers	vLLM/TGI/Ollama	Replaceable model execution
Async queue	Cloudflare Queues	NATS JetStream/RabbitMQ	Delivery mechanism; event truth elsewhere
Durable orchestration	Cloudflare Workflows	Temporal	Workflow execution state
Feature flags	Flagship	Unleash/OpenFeature provider	Evaluation provider; flag definitions governed by app